

1. Introducció

1.1. Contextualització del projecte

El projecte “*Mòdul analític de dades de software senyalització digital*” que és dirigit per en Daniel Novoa i assessorat per en Carles Farré, es desenvolupa en el marc del treball de fi de grau del grau d'Enginyeria Informàtica de la Facultat d'Informàtica de Barcelona, Universitat Politècnica de Catalunya, en la menció d'Enginyeria del Software. El treball es realitza en la modalitat B, és a dir, com a projecte realitzat juntament amb una empresa mitjançant el Conveni de Cooperació Educativa de la UPC, més concretament amb l'empresa *Waapiti*.

Waapiti és una empresa especialitzada en donar solucions de senyalització digital [1], que fa ús de les pantalles digitals com vindrien a ser monitors o pantalles LED per a la visualització de continguts de forma dinàmica, diferenciant-se de la senyalització tradicional, que fa ús de cartells estàtics. A més, el software de l'empresa permet fer una gestió dels continguts en totes les pantalles que estiguin connectades a la xarxa, i d'aquesta manera poder actualitzar-les remotament. A més de la gestió del contingut i les pantalles, el software també permet tenir un control dels usuaris i clients de l'empresa i de tots els elements relacionats amb ells.

En el cas d'aquest projecte, dintre de l'enginyeria del software es centrarà en les tecnologies de desenvolupament web i més concretament en el backend. Com a framework es farà ús de *NestJS* [2], amb el llenguatge *Typescript* [3], i per la infraestructura es farà ús dels serveis cloud d'*AWS* [4]. Aquestes tecnologies són les mateixes de les quals s'utilitzen actualment a l'empresa de *Waapiti*.

Per facilitar l'enteniment del projecte, solament es definirà les parts del software que tinguin una relació directa amb l'actual projecte, com vindrien a ser la gestió de les dades generades per les pantalles i els usuaris.

1.2. Conceptes

En aquest apartat es farà una explicació de la terminologia relacionada amb la senyalització digital i més concretament amb la terminologia utilitzada a *Waapiti* per així facilitar la lectura i enteniment del treball.

- **Player:** Un player consisteix en un dispositiu que pot ser intern o extern, que permet a les pantalles reproduir continguts en les pantalles. Consisteix en un ordinador en miniatura que connecta la pantalla a la plataforma de l'empresa per poder gestionar i reproduir els continguts.
- **Data lake:** Un data lake és un repositori centralitzat que permet l'emmagatzematge de dades estructurades i no estructurades a qualsevol escala. Permet guardar aquestes dades sense que s'hagin de modificar ni estructurar.

- **Playlog:** Un playlog consisteix en la informació provinent dels players que indica quin contingut s'està reproduint en aquell instant. Entre la informació que conté, té la data, juntament amb l'hora, minuts i segons), el player i el contingut reproduït.
- **Connexió:** Les connexions consisteixen en dades provinents dels players un cop cada hora, que ens indiquen si el player corresponent està connectat a la plataforma o no.
- **Audit Trail:** L'audit trail consisteix en l'històric del conjunt d'accions que es realitzen sobre la plataforma. Entre la informació que conté, està l'acció, la data en què s'ha realitzat, sobre quina entitat s'ha realitzat i qui l'ha realitzat.
- **Web socket:** Un web socket és la tecnologia que permet la comunicació bidireccional entre la plataforma i els players, i d'aquesta manera permet l'intercanvi d'informació entre ells.
- **SAT:** SAT o Servei d'Atenció Tècnica és el departament de Waapiti que s'encarrega de donar suport tècnic relacionat amb els players als clients de la plataforma.

1.3. Descripció del problema

Amb aquest projecte es vol solucionar tres problemes relacionats entre si que presenta l'empresa actualment:

- **La ineficiència en l'emmagatzematge de les dades provinents dels players:** Actualment, per a l'emmagatzematge de les dades provinents dels players es fa ús de *DynamoDB* [5], que és una base de dades NoSQL i que forma part dels serveis cloud d'*AWS*. Aquestes dades no són fàcilment accessibles per la plataforma, ja que no estan ni ordenades ni estructurades, cosa que fa que accedir a aquestes dades tingui un cost elevat per l'empresa. Un altre punt important, que empitjora el problema és que el nombre de players va augmentant de manera contínua, i per tant també ho fa la quantitat de dades i el cost per poder-les emmagatzemar i llegir.
- **La falta de dades de les accions realitzades en plataforma:** Un altre problema important que presenta l'empresa és la falta d'obtenció de dades de les accions que es realitzen dintre de la plataforma de l'empresa. Això consisteix en un problema, ja que no es guarda constància dels canvis que pugui haver provocat algun usuari en algun player o qualsevol altre element de la plataforma. Això provoca que si hi ha hagut algun canvi important o alguna equivocació per part d'algun usuari no es podrà solucionar de la manera més eficient possible, ja que es desconixerà qui l'ha provocat. A més, també afecta de manera negativa degut el fet que no es pugui obtenir l'historial d'accions provocades sobre alguna entitat de la plataforma i d'aquesta manera poder veure totes les accions realitzades sobre ella.

- **Obtenció i anàlisi de les dades:** Un altre problema important relacionat amb els dos anteriors és la dificultat en l'obtenció de les dades i la falta d'eines d'anàlisi que ens permetin analitzar-les. Com s'ha dit en el primer punt, a causa de com estan emmagatzemades les dades fa que sigui molt difícil obtenir-les per poder-les utilitzar. Actualment, l'empresa no té cap manera d'extreure conclusions de l'ús que es fa de la plataforma. Per exemple, no hi ha cap eina que ens permeti obtenir el contingut que més vegades s'ha reproduït ni tampoc el nombre de vegades que s'ha reproduït un contingut en específic en un player. Això provoca que ni els usuaris i ni la mateixa empresa pugui tenir un control de què succeeix en la plataforma ni en els players. A més, juntament amb el problema anterior, tampoc tenim cap manera d'obtenir, per exemple, quin és l'usuari que més contingut puja a un player o qui més canvis realitza.

En resum, la plataforma manca d'un sistema d'emmagatzematge eficient que permeti guardar totes les dades necessàries a gran escala i d'un sistema que ens permeti obtenir-les i fer l'anàlisi d'aquestes dades.

1.4. Stakeholders

- **SAT:** Aquest departament serà el principal beneficiari intern d'aquest projecte. Això és degut al fet que aquest actor és qui dona suport als clients de la plataforma i farà ús del projecte per poder millorar l'atenció al client.
- **Product Owner:** És l'encarregat d'obtenir el màxim valor del producte i d'assegurar una rendibilitat de la inversió. Representa als clients dintre de l'equip i és la cara visible del projecte davant dels clients. És l'encarregat de decidir que és el que es farà en cada sprint, encara que també hi participarà el desenvolupador del projecte.
- **Desenvolupador:** És qui s'encarregarà de dissenyar i implementar tot el software del projecte, de documentar-lo i de recollir tots els requisits provinents dels diferents actors del projecte. Aquest desenvolupador és l'autor del TFG i és qui dissenyarà i implementarà tot el software necessari.
- **Clients:** Seran els beneficiaris externs principals d'aquest projecte, ja que gràcies a aquest projecte podran obtenir un millor control dels seus usuaris i players.
- **Director del projecte:** El director del projecte, a més de ser desenvolupador sènior de *Waapiti*, s'encarregarà de supervisar la correcció en la implementació del codi del software i proposarà millores en el disseny.
- **Ponent del projecte:** El ponent s'encarregarà de l'assessorament acadèmic del projecte i de la seva correcció.

8. Especificació de requisits

A continuació s'especificaran els requisits esmentats anteriorment de manera detallada, indicant les condicions que han de satisfer per poder procedir amb la seva acceptació.

8.1. Requisits funcionals

Els requisits funcionals són els requisits que defineixen les funcionalitats del sistema. Per poder fer-los més clars, s'han utilitzat històries d'usuari que descriuen les funcionalitats des del punt de vista dels usuaris.

8.1.1. Històries d'usuari

Les històries d'usuari són una representació del requisit escrita des del punt de vista de l'usuari en un llenguatge simple i fàcil d'entendre. Les històries d'usuari segueixen el següent patró:

- Com a **[Rol]**, vull **[Acció]** per **[Benefici]**.

A més, per cada història d'usuari també s'han definit els criteris d'acceptació, que ens permeten saber que la història d'usuari ha estat finalitzada.

Història d'usuari	H1: Emmagatzemar playlogs
Com a	Qualsevol tipus d'usuari
Vull	Que el sistema guardi els playlogs dels players
Per	Poder tenir constància dels continguts reproduïts en els diferents players
Criteris d'acceptació	• Cada playlog té una data, client i player assignat.

*Taula 8.1. Història d'usuari 1
Font: Elaboració pròpia*

Història d'usuari	H2: Emmagatzemar connexions
Com a	Qualsevol tipus d'usuari
Vull	Que el sistema guardi les connexions dels players
Per	Poder tenir constància de quins players estan connectats a cada hora
Criteris d'acceptació	• Cada connexió té una data, client i player assignat. • L'estat de la connexió solament pot ser connectat o no connectat.

*Taula 8.2. Història d'usuari 2
Font: Elaboració pròpia*

Història d'usuari	H3: Emmagatzemar accions de plataforma
Com a	Qualsevol tipus d'usuari
Vull	Que el sistema guardi les accions realitzades sobre la plataforma
Per	Poder tenir constància de quines accions s'han realitzat sobre la plataforma
Criteris d'acceptació	• Cada acció té una data, client, usuari i player assignat. • Totes les accions són realitzades sobre una entitat de la plataforma.

Taula 8.3. Història d'usuari 3
Font: Elaboració pròpia

Història d'usuari	H4: Llistar playlogs
Com a	Qualsevol tipus d'usuari amb permisos
Vull	Poder llistar els diferents playlogs del sistema
Per	Poder saber quins continguts s'han reproduït en els diferents players del sistema
Criteris d'acceptació	• Els playlogs que podrà veure l'usuari dependrà dels permisos que té. • S'ha de poder veure tots els detalls del playlog.

Taula 8.4. Història d'usuari 4
Font: Elaboració pròpia

Història d'usuari	H5: Llistar connexions
Com a	Qualsevol tipus d'usuari amb permisos
Vull	Poder llistar les diferents connexions dels players del sistema
Per	Poder saber quins players estan connectats i quins no
Criteris d'acceptació	• Les connexions que podrà veure l'usuari dependrà dels permisos que té. • Les connexions solament podran tenir dos estats: connectat o no connectat.

Taula 8.5. Història d'usuari 5
Font: Elaboració pròpia

Història d'usuari	H6: Llistar accions de plataforma
Com a	Qualsevol tipus d'usuari amb permisos
Vull	Poder llistar les diferents accions que es realitzen sobre les diferents entitats de la plataforma
Per	Poder saber quines accions s'han realitzat en la plataforma al llarg del temps
Criteris d'acceptació	• Les accions que podrà veure l'usuari dependrà dels permisos que té. • Totes les accions han d'estar relacionades amb una entitat de la plataforma.

Taula 8.6. Història d'usuari 6
Font: Elaboració pròpia

8.2. Requisits no funcionals

Com s'ha dit anteriorment, els requisits no funcionals són els requisits que ens permeten establir la qualitat que ha de tenir els software a desenvolupar.

En aquest apartat, es defineix cada requisit amb més detall indicant perquè és necessari i quin criteri ha de satisfer per poder establir que s'ha complert.

Requisit	12a. Velocitat i latència
Descripció	El sistema ha de respondre ràpidament quan l'usuari realitzi alguna acció.
Justificació	L'usuari no pot esperar massa en rebre una resposta del sistema, ja que disminueix la confiança en el sistema i genera frustració.
Acceptació	El sistema respon a qualsevol acció de l'usuari en menys de 5 segons.

Taula 8.7. Requisit no funcional 1
Font: Elaboració pròpia

Requisit	12d. Disponibilitat i fiabilitat
Descripció	El sistema ha d'estar sempre disponible en tot moment, excepte alguna puntualitat, com pot ser una actualització.
Justificació	La plataforma de <i>Waapiti</i> s'utilitza durant les 24 hores del dia i, per tant, ha d'estar disponible per tots els usuaris en tot moment.
Acceptació	Qualsevol usuari ha de poder accedir a la plataforma durant qualsevol moment del dia. Excepte en les actualitzacions programades.

Taula 8.8. Requisit no funcional 2
Font: Elaboració pròpia

Requisit	12f. Capacitat
Descripció	El sistema ha de poder ser capaç de suportar un nombre il·limitat de connexions al mateix temps.
Justificació	El sistema té un gran nombre d'usuaris que han de poder accedir sense cap problema independentment del nombre d'usuaris connectats.
Acceptació	La plataforma escala de forma dinàmica amb els canvis de connexions simultànies.

Taula 8.9. Requisit no funcional 3
Font: Elaboració pròpia

Requisit	12g. Escalabilitat
Descripció	El sistema ha de poder créixer en capacitat quan el volum de dades augmenti.
Justificació	S'espera que en <i>Waapiti</i> el nombre d'usuaris augmenti i, per tant, també augmentarà el nombre de dades produïdes per ells. El sistema de poder escalar amb aquest augment de dades.
Acceptació	El sistema escala de manera automàtica a la vegada que van augmentant les dades.

Taula 8.10. Requisit no funcional 4
Font: Elaboració pròpia

Requisit	15a. Accés
Descripció	Les funcionalitats del projecte han de ser solament accessibles als usuaris amb els permisos corresponents.
Justificació	Cada usuari de <i>Waapiti</i> té accés a certes funcionalitats, depenent del tipus d'usuari que sigui. Un usuari no pot accedir a funcionalitats que no tingui.
Acceptació	Els usuaris sense permís no poden accedir a les dades de les quals no tenen permís.

Taula 8.11. Requisit no funcional 5

Font: Elaboració pròpia

Requisit	15c. Privacitat
Descripció	La informació confidencial dels usuaris ha d'estar protegida per tal de garantir la seva privacitat.
Justificació	Cada usuari de <i>Waapiti</i> té dades confidencials que solament pot veure el propi usuari.
Acceptació	Els usuaris solament poden observar les dades confidencials del seu usuari i de ningú més.

Taula 8.12. Requisit no funcional 6

Font: Elaboració pròpia

8.3. Model conceptual del domini

En la figura 8.1 es pot observar el diagrama del model conceptual del domini d'aquest projecte.

En aquest model es pot observar que un client pot tenir diferents usuaris (*Users*) en la plataforma, i que aquests usuaris són qui realitzen les accions en la plataforma (*Audit trail*). Per altra banda, els clients poden tenir diferents players que reproduïxen continguts (*Playlog*) i que estan connectats per un cert temps (*Connection*).

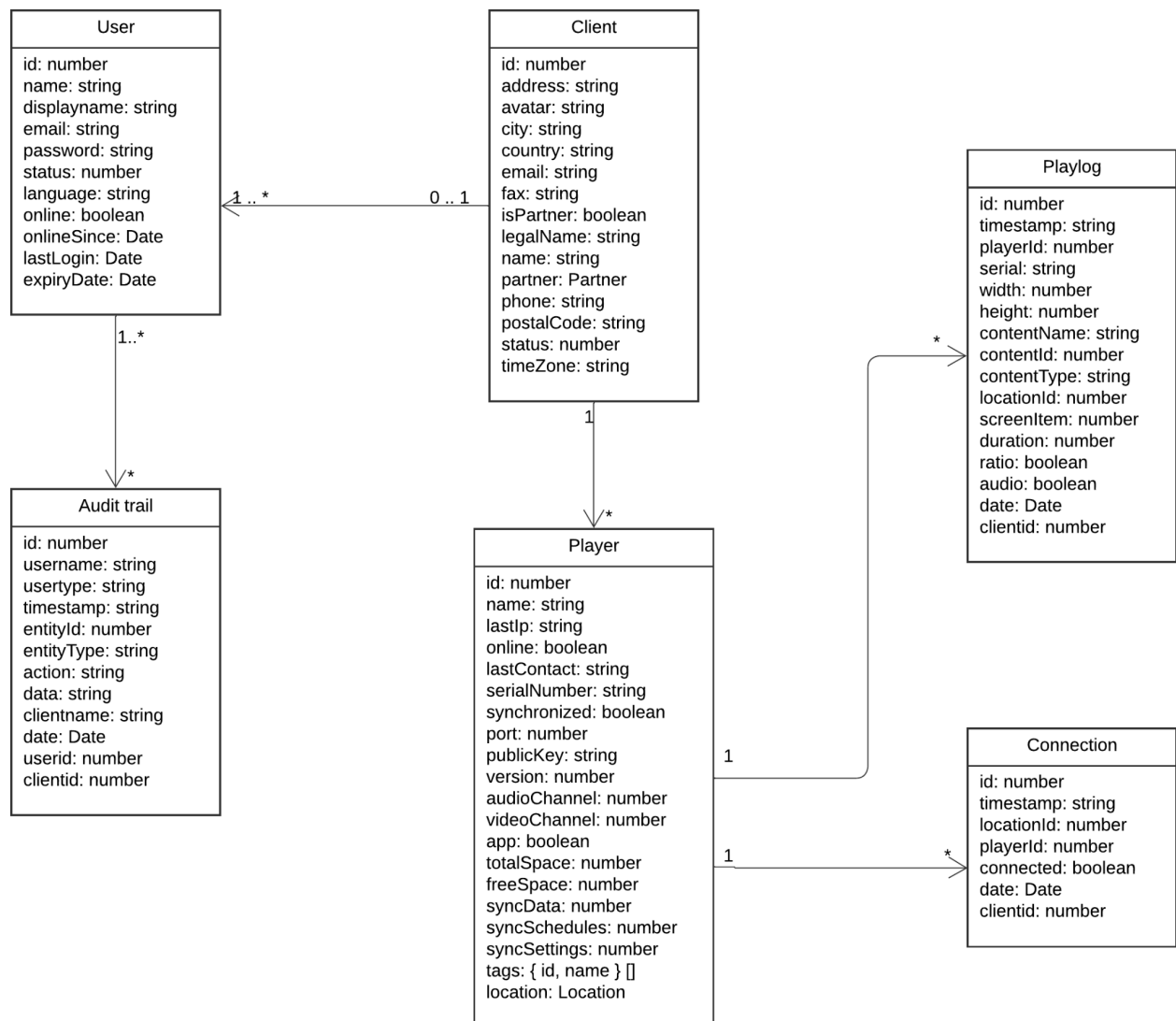


Figura 8.1. Diagrama conceptual del domini

Font: Elaboració pròpia

9. Arquitectura

9.1. Arquitectura lògica

En el nostre projecte la plataforma de *Waapiti* és un servei web que segueix l'arquitectura client-servidor.

En aquest tipus d'arquitectura el client és qui realitza les peticions a un servidor, que les processa i retorna la resposta al client. En el nostre cas, el client és l'usuari de la plataforma i el servidor és el software que s'encarrega de processar les peticions de l'usuari.

La comunicació entre els dos components es realitza a través d'internet mitjançant el protocol HTTP [26] i el model de REST API.

Pel model client-servidor s'utilitza l'arquitectura de 3 capes, on el sistema es divideix en 3 capes, la capa de presentació, la capa de negoci i la capa de dades.

- **Capa de presentació:** És la capa que s'encarrega de mostrar la informació a l'usuari. En aquest projecte, és la interfície visual de la plataforma de *Waapiti*.
- **Capa de negoci:** Aquesta capa és el nucli de l'aplicació, que s'encarrega de processar tota la informació. En el nostre context, és el software que s'encarrega de processar les peticions de l'usuari.
- **Capa de dades:** És la capa que s'encarrega d'emmagatzemar totes les dades. En el nostre cas vindrien a ser la base de dades i els serveis d'emmagatzament d'AWS.

9.2. Arquitectura física

En el cas de la plataforma de *Waapiti*, l'arquitectura física no es correspon exactament a l'arquitectura lògica, ja que tant la capa de negoci com la capa de dades es divideixen en diversos serveis diferents. L'objectiu d'això és millorar la seguretat, l'escalabilitat i la disponibilitat del sistema.

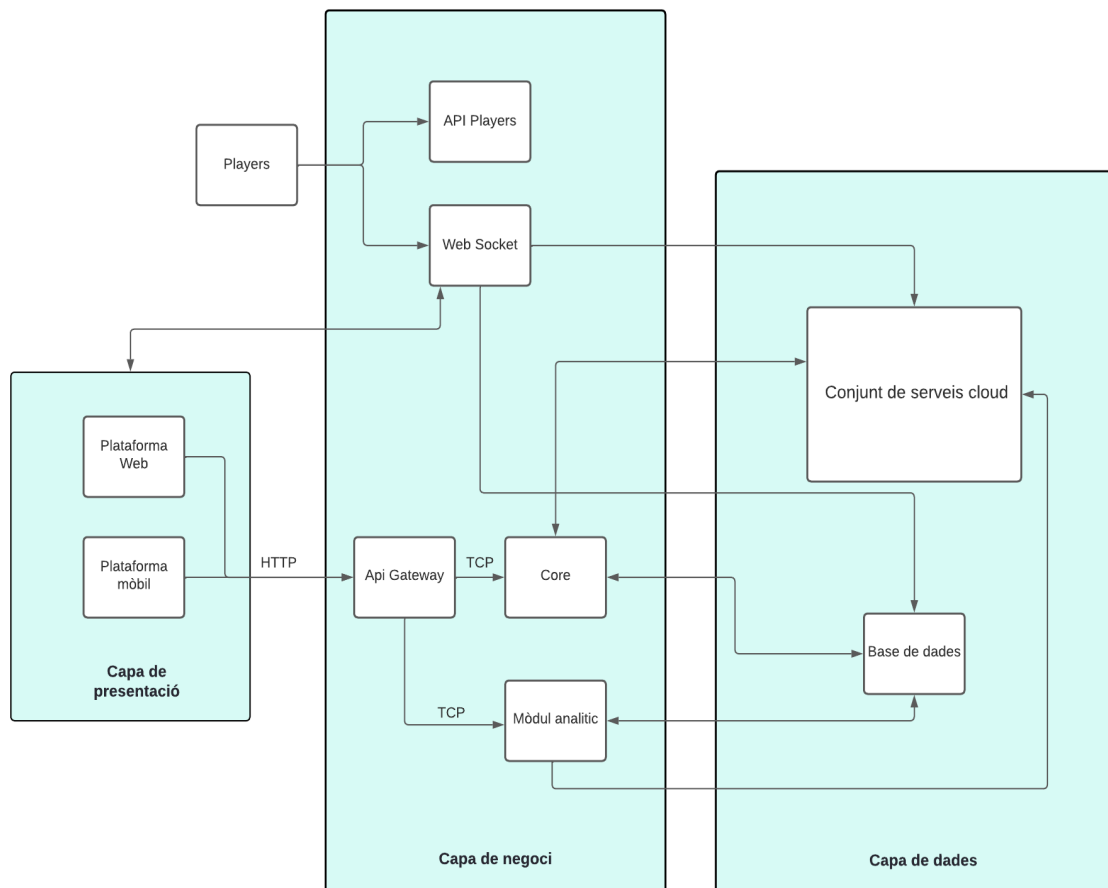


Figura 9.1. Arquitectura del sistema
Font: Elaboració pròpia

En la figura 9.1 es pot observar l'esquema de l'arquitectura de tot el sistema. Es pot observar que les diferents capes que formen el sistema es componen de diversos serveis que interactuen entre si.

9.2.1. Capa de presentació

En el cas d'aquesta capa, es pot observar de què està formada per la interfície de la plataforma web i la plataforma mòbil de *Waapiti*. A més, com s'ha indicat anteriorment, la capa de presentació es comunica amb la capa de negoci, més concretament amb l'API Gateway, mitjançant el protocol HTTP.

9.2.2. Capa de negoci

Com es pot observar en el diagrama la capa de negoci està formada per cinc components. A més, es pot observar de què tres components estan relacionats de manera directa o indirecta amb la capa de presentació, és a dir, que interaccionen amb els usuaris. Aquests components vindrien a ser, l'Api Gateway, el Core i el mòdul analític.

Com es pot veure, l'únic component que interactua de manera directa amb la capa de presentació és el mòdul Api Gateway. Això és degut de què es segueix el patró de disseny de microserveis Api Gateway [27]. Aquest patró de disseny permet que solament un component sigui qui interactuï entre els usuaris i els serveis que proporciona la plataforma. L'ús d'aquest patró proveeix diversos avantatges al sistema, com poden ser:

- Augmenta la seguretat del sistema al reduir les portes d'enllaç amb l'exterior, ja que l'única manera d'interactuar amb els microserveis és a través de l'Api Gateway.
- Dona la capacitat de manegar caigudes parcials del sistema, manegant quins serveis estan en funcionament en tot moment.
- És solament l'Api Gateway que s'encarrega de l'autorització i el SSL amb el client.

En la següent figura 9.2 es pot observar com funciona el patró Api Gateway:

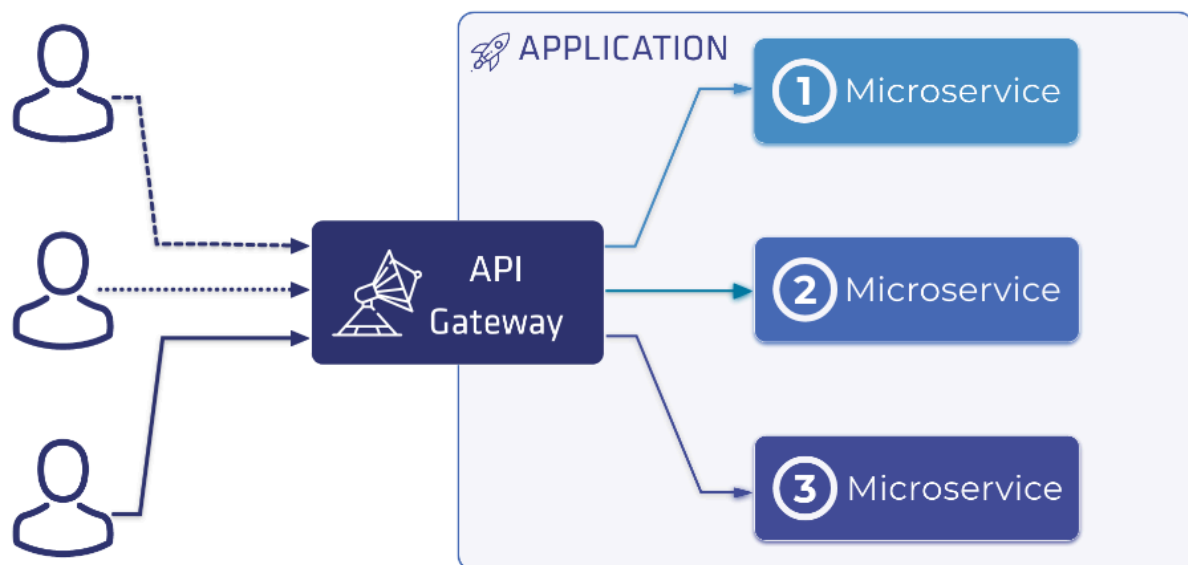


Figura 9.2. Patró Api Gateway
Font: velog.io [28]

Com es pot observar en la figura 9.1 la comunicació entre el servei de l'Api Gateway és mitjançant el protocol de comunicació TCP [29]. Cadascun d'aquests serveis tenen funcionalitats específiques:

- **Servei Api Gateway:** Aquest servei s'encarrega de la gestió, autenticació i autorització dels usuaris de la plataforma. Defineix totes les rutes de l'API REST i processa les peticions per poder enviar-les al servei Core i al mòdul analític.
- **Servei Core:** Aquest servei s'encarrega de realitzar la lògica de negoci de la plataforma, seria el nucli principal de la plataforma. Principalment, interactua amb la base de dades, encara que també interactua en menor mesura amb els serveis cloud.
- **Mòdul analític:** Aquest servei és el servei que s'ha començat a desenvolupar durant el desenvolupament d'aquest TFG. El mòdul analític té com a funció principal obtenir i processar les dades dels playlogs, les connexions i les accions en plataforma. Principalment, aquest servei interactua amb els serveis cloud, encara que també interactua en menor mesura amb la base de dades.

Com hem dit anteriorment, en la capa de negoci tenim cinc components, i a part dels components que interactuen amb els usuaris tenim dos components que interactuen amb els players, aquest vindrien a ser l'Api Player i el Web Socket.

- **Api Player:** Aquest servei s'encarrega de comunicar-se amb els players rebent tota la informació corresponent a aquest. La informació que poden rebre poden ser l'estat del player, els continguts que reproduïx, els calendaris en què s'han de reproduir aquests continguts, etc. Aquesta informació es processa i s'emmagatzema.
- **Web Socket:** Aquest servei s'encarrega de realitzar la comunicació entre la plataforma i els players, i d'aquesta manera permet l'intercanvi d'informació entre ells. Processa les dades provinents dels players i les emmagatzema en la base de dades i en els serveis cloud.

Es pot observar que la capa de negoci rep informació de dues fonts diferents, que són la plataforma, és a dir, els usuaris, i els players. Després d'això, s'interactua amb aquesta informació i es comunica amb la capa de dades.

9.2.3. Capa de dades

La capa de dades és la capa que s'encarrega d'emmagatzemar les dades del sistema. Com es pot observar en la figura 9.1, la capa de dades està formada pels serveis cloud AWS i la base de dades.

- **Base de dades:** La base de dades utilitzada és una base de dades MySQL que s'encarrega d'emmagatzemar totes les dades necessàries per al funcionament del

sistema. Aquesta base de dades està implementada en el servei RDS [30], que és un servei de base de dades relacional distribuït d'AWS.

- **Conjunt de serveis cloud AWS:** L'objectiu d'aquests serveis és formar un data lake on s'emmagatzemi tots els històrics per poder processar-los en el mòdul analític. Aquest data lake està format per diversos serveis:
 - **Amazon S3:** Aquest és el servei d'emmagatzament d'objectes d'AWS. En aquest projecte s'encarrega d'emmagatzemar documents JSON que contenen els playlogs, les connexions i l'audit trail. Dintre de cada bucket de S3 els documents s'emmagatzemen per carpetes depenent de la data i per subcarpetes depenent del client i/o usuari.
 - **DynamoDB:** Aquest servei és una base de dades no relacional d'AWS. En el nostre cas s'encarrega d'emmagatzemar els playlogs en temps real. Els playlogs són accessibles mitjançant la Composite Key, que està formada per la Partition Key, que vindria a ser l'identificador del player, i la Sort Key, que vindria ser la data i hora.

10. Technologies

En aquest apartat es farà un breu resum de les tecnologies i frameworks utilitzades durant el desenvolupament d'aquest projecte.

Totes les tecnologies utilitzades durant el desenvolupament del TFG han contribuït de manera molt positiva en l'aprenentatge de l'autor, ja que es tracten de tecnologies molt utilitzades en el desenvolupament web.

- **NestJS [2]:** Aquest framework és un framework de *NodeJS* que té com a funció principal la creació d'aplicacions de backend. En aquest projecte, s'ha utilitzat aquest framework per desenvolupar els diferents serveis que interactuen entre si. Aquest framework permet la creació d'una API REST segura i complexa d'una manera més fàcil que fent ús directe de *NodeJS*.
- **TypeScript [3]:** Aquest llenguatge de programació que està construït a un nivell superior de *JavaScript*, és a dir, que estén la sintaxi de *JavaScript*. S'ha decidit la utilització d'aquest llenguatge, i no *JavaScript*, ja que és el llenguatge per defecte per *NestJS* i perquè gràcies al seu tipatge estàtic permet la detecció d'errors en temps de compilació.
- **Infraestructura AWS [4]:** És el conjunt de serveis de computació en el núvol que permet la creació d'aplicacions escalables i altament disponibles. En aquest projecte, s'ha utilitzat per desplegar la plataforma web i l'API REST. A més, també s'ha utilitzat per emmagatzemar les dades històriques de la plataforma. S'ha decidit la utilització d'AWS i no altres serveis cloud pel seu baix preu i la seva utilització prèvia per part de l'empresa.

11. Implementació

En aquest apartat es farà una descripció de la implementació del sistema desenvolupat durant el projecte. L'explicació de la implementació està separada en dues parts: la infraestructura i el backend, que són les parts desenvolupades durant el projecte.

Per fer més fàcil la comprensió d'aquest apartat s'ha fet ús de captures de pantalles de les diferents parts que intervenen en el desenvolupament del projecte.

11.1. Infraestructura

En aquest subapartat es farà una explicació sobre el desenvolupament de la infraestructura del projecte i una explicació de totes les parts que la componen.

Com s'ha anat indicant en apartats anteriors, per la creació de la infraestructura del projecte s'ha fet ús d'Amazon Web Services (AWS). El nostre objectiu principal era desenvolupar la infraestructura necessària que ens permetés poder gestionar, obtenir i emmagatzemar les dades dels playlogs, les connexions i les accions en plataforma (audit trail). Per tal d'obtenir això s'ha fet ús de diferents serveis d'AWS que ens han permès desenvolupar un *data lake*.

Un *data lake* consisteix en un dipòsit de dades que poden ser emmagatzemades en el seu format natural. Com es pot veure en la figura 11.1, les dades a emmagatzemar poden ser dades estructurades de bases de dades relacionals, semiestructurades, com podrien ser fitxers JSON, que és el nostre cas, dades no estructurades, com poden ser PDFs, i dades binàries, com podrien ser vídeos. Una de les principals característiques del *data lake* és permetre dur a terme l'anàlisi de les dades emmagatzemades en aquest dipòsit. A més, el *data lake* pot ser establert en local o en el núvol, com seria el nostre cas.

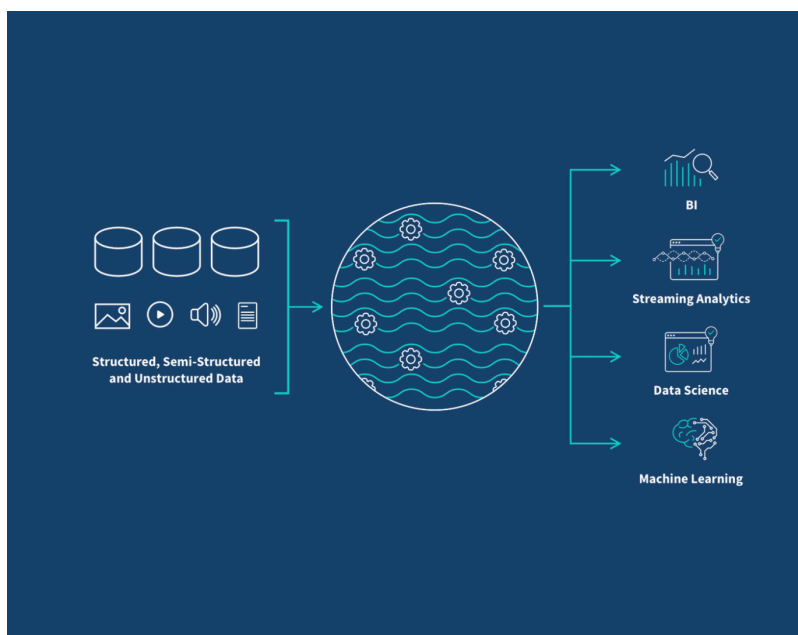


Figura 11.1. Esquema d'un data lake

Font: qlik.com [31]

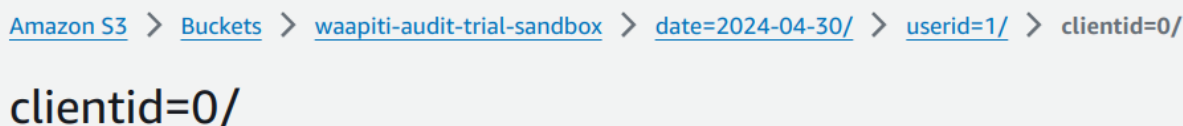
Com s'ha indicat abans per desenvolupar tota la infraestructura s'han utilitzat serveis d'AWS i ara procedirem a fer una explicació dels serveis utilitzats per així poder conèixer com funcionen i com interactuen entre si.

11.1.1. Serveis d'AWS

Els serveis utilitzats tenen diferents funcions principals que en conjunt permeten crear la infraestructura necessària per poder gestionar les dades la plataforma. Les funcions principals són les següents: emmagatzematge, lectura de dades i transmissió de dades.

- **Emmagatzematge:** Aquesta funció és una funció important per la infraestructura, l'emmagatzematge de les diferents dades és el que ens permet tenir les dades necessàries per poder-les utilitzar. Els següents serveis són els que s'encarreguen d'implementar aquesta funció:
 - **Amazon S3:** El servei d'Amazon S3 és un servei necessari per poder desenvolupar un *data lake*, ja que aquest servei ens permet emmagatzemar qualsevol element que es necessiti.
En el nostre cas, s'ha fet servir aquest servei per poder guardar fitxers JSON que contenen les variables de les dades de la plataforma. Cadascun dels tres tipus de dades s'emmagatzemen en buckets diferents i dintre de cada bucket els documents es guarden en diferents subcarpetes depenent de la data, el client i en el cas de l'audit trail, també l'usuari, com es pot veure en la figura 11.2. L'emmagatzament s'ha fet d'aquesta manera, ja que com s'explicarà més endavant, fa que la lectura de dades sigui més eficient.

A més, dels tres buckets utilitzats per emmagatzemar els tres tipus de dades diferents també s'ha creat un bucket que conté els resultats de les lectures de dades dels buckets principals.



The image shows a breadcrumb navigation path in an Amazon S3 console. The path is: [Amazon S3](#) > [Buckets](#) > [waapiti-audit-trial-sandbox](#) > [date=2024-04-30/](#) > [userid=1/](#) > [clientid=0/](#). Below this path, the text 'clientid=0/' is displayed in a larger font.

*Figura 11.2. Subcarpetes del bucket d'audit trail
Font: Elaboració pròpia*

- **Amazon DynamoDB:** El servei de DynamoDB és un servei de base de dades NoSQL. En el cas d'aquesta base de dades està formada per taules, que a la seva vegada estan formades per elements que són pars clau-valor. La clau primària d'aquests elements s'anomena *Composite Key*, ja que està formada per dos claus, que són la *Partition Key* i la *Sort Key*. S'utilitza aquest conjunt de dos claus per poder identificar en quina partició es troba el valor que busquem, ja que les taules poden estar dividides en una o més particions.

En el cas d'aquest projecte, l'ús de DynamoDB serà per l'emmagatzament en temps real dels playlogs. Per poder identificar els diferents elements de la taula s'utilitzarà com a clau primària l'identificador del player, com a *Partition Key*, i la data i l'hora, com a *Sort Key*.

- **Lectura de dades:** Aquesta funció és necessària i important tant per la infraestructura com el projecte en conjunt, ja que sense la lectura de les dades, el mòdul analític no podria dur a terme les funcions per les quals ha sigut desenvolupat. Per poder obtenir aquesta funció s'han fet ús dels següents serveis:
 - **Amazon Athena:** El servei d'Amazon Athena és un servei que permet realitzar consultes en SQL sobre dades emmagatzemades en Amazon S3. Per poder dur a terme l'anàlisi de les dades emmagatzemades, és necessari que els documents guardats en S3 siguin en format CSV, JSON o Parquet. En la figura 11.3 es pot veure de manera esquemàtica com funciona aquest servei. Aquest servei, al ser un servei sense servidor fa que no sigui necessari administrar la infraestructura quan hi ha un augment en el conjunt de dades i usuaris. A més, per poder obtenir les dades d'una manera ràpida, Athena realitza les consultes en paral·lel.

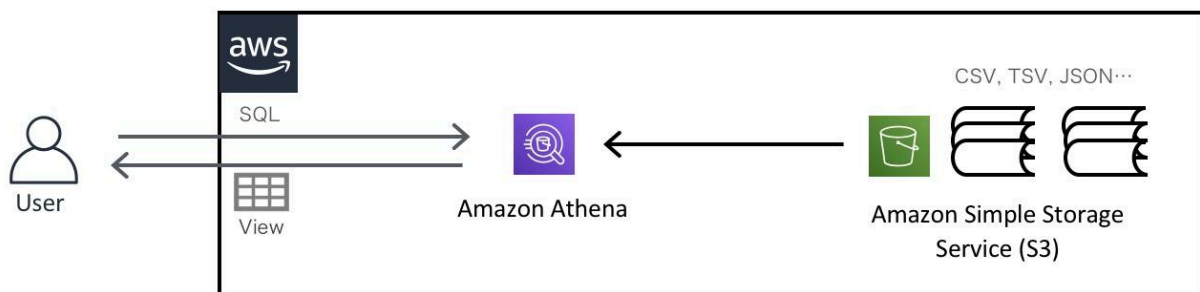


Figura 11.3. Esquema del funcionament d'Athena
Font: tdi.co.jp [32]

En el desenvolupament d'aquest projecte l'ús d'Amazon Athena ha sigut una de les parts més importants, ja que permetia poder llegir totes les dades que teníem en els nostres buckets. Per cada bucket s'ha creat una taula corresponent amb les columnes corresponents. Com hem indicat abans els documents que s'emmagatzemen en S3 estan en el format JSON, que és un dels formats que pot llegir Athena. A més, s'han creat particions, que juntament amb les subcarpetes d'Amazon S3, fan que sigui més eficient la lectura de dades de les diferents taules.

- **AWS Glue:** El servei d'AWS Glue és un servei d'integració de dades sense servidor que ens permet la detecció, preparació, migració i integració de les dades provinents de diferents orígens.

En el cas del desenvolupament d'aquest projecte hem fet ús de la funció d'AWS Glue Data Catalog. L'ús d'aquesta ha sigut necessària per poder emmagatzemar

les metadades de les dades del projecte, ja que el Glue Data Catalog funciona com un repositori que conté totes les metadades respecte a on estan emmagatzemades les dades i de com estan definides les diferents taules, incloent-hi les particions de cada taula. D'aquesta manera treballa en conjunt amb Athena perquè pugui llegir les dades des del S3 de manera correcta.

- **Transmissió de dades:** Un altre punt important és la transmissió de dades des d'on s'originen al lloc d'emmagatzematge. Perquè sigui la transmissió de dades la més ràpida possible s'han fet ús de dos serveis d'Amazon, que ens permeten realitzar una transmissió en quasi temps real.
 - **Kinesis Data Streams:** Kinesis Data Streams és un servei de processament i ingesta de dades dirigit especialment al streaming de dades. Com es pot observar en la figura 11.4 el stream llegeix les dades de diferents generadors (*Producers*) i les envia als consumidors (*Consumers*). Les dades transmeses pels generadors es troben disponibles en el stream en mil·lisegons i els consumidors podran fer ús d'aquestes dades en l'ordre d'arribada. A més, com es pot veure en la figura 11.4 el stream pot estar dividit en diverses particions (*Shards*). El nombre de particions que contengui un stream dependrà de la quantitat de dades que rebrà normalment. Dintre de cada partició hi haurà el registre de dades, que es compon per un número de seqüència, una clau de partició i la dada en si. Cada registre de dades es manté dintre del stream com a mínim fins a les 24 hores, i si es configura, fins a un màxim de 365 dies.

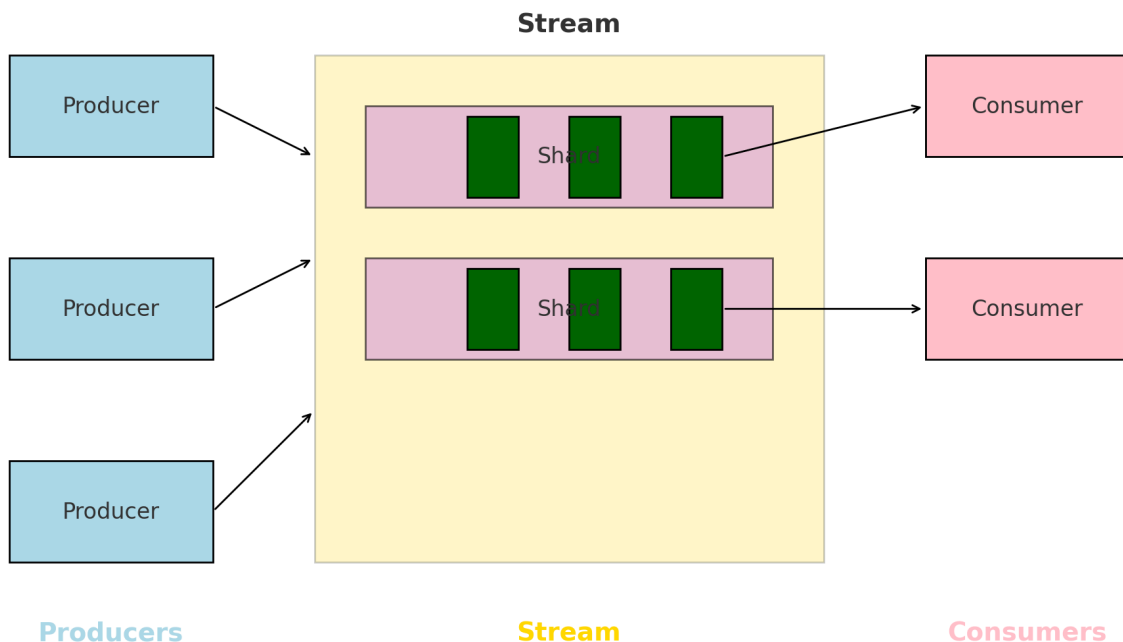


Figura 11.4. Arquitectura de Kinesis Data Streams
Font: Elaboració pròpia

En el desenvolupament d'aquest projecte s'ha fet ús de Kinesis Data Streams per poder transmetre les dades dels playlogs i l'audit trail, ja que en el cas de les connexions s'ha decidit emmagatzemar-les directament a S3. Això és degut al fet que Kinesis Data Streams té un preu per hores i en el cas de les connexions solament s'emmagatzemen un cop cada hora. En el cas dels generadors, aquests serien els diferents microserveis de NestJS que tenen la informació sobre els playlogs o les accions en plataforma. En el cas dels playlogs el generador el microservei WebSocket, que és qui s'encarrega de rebre la informació dels players. Mentre que l'audit trail té com a generador el Core, que és el microservei central de la plataforma. Per part dels consumidors tenim el servei de Kinesis Data Firehose, que s'encarrega de guardar les dades en el S3. A més, en el nostre cas, cada un dels streams, fa ús d'un shard, ja que actualment el nombre de dades no supera la capacitat d'aquests shards.

- **Kinesis Data Firehose:** El servei de Kinesis Data Firehose té com a funció principal la càrrega de grans quantitats de dades en magatzems de dades. La unitat principal d'aquest servei el Delivery Stream, que és on s'envia les dades des dels generadors. Aquestes dades que s'envien poden ser com a màxim de 1000 KB. La càrrega d'aquestes dades en els magatzems de dades es pot realitzar per tamany o per temps, és a dir, quan el Delivery Stream arriba a un cert volum de dades o quan passa un cert temps de la càrrega anterior.

Com hem dit anteriorment, en aquest projecte hem fet ús de Kinesis Data Stream per emmagatzemar les dades dels playlogs i l'audit trail en els seus buckets corresponents de S3. A més, s'ha fet ús d'una de les funcionalitats que té Kinesis Data Firehose respecte al servei de S3, que és el particionament dinàmic. Aquesta funció ens permet crear particions en S3, i d'aquesta manera crear subcarpetes, respecte a unes claus predefinides. També s'ha decidit que per les dues instàncies de Kinesis Data Firehose, la dels playlogs i l'audit trail, es faria la càrrega de les dades cada 60 segons o quan s'arribés a un tamany de dades de 5 MB.

11.1.2. AWS Cloud Development

Per poder desenvolupar tota la infraestructura necessària per al projecte s'ha fet ús de l'AWS Cloud Development, també anomenat AWS CDK. L'AWS CDK és el framework d'AWS que permet la construcció d'aplicacions en el núvol, ja que permet la construcció de la infraestructura mitjançant codi.

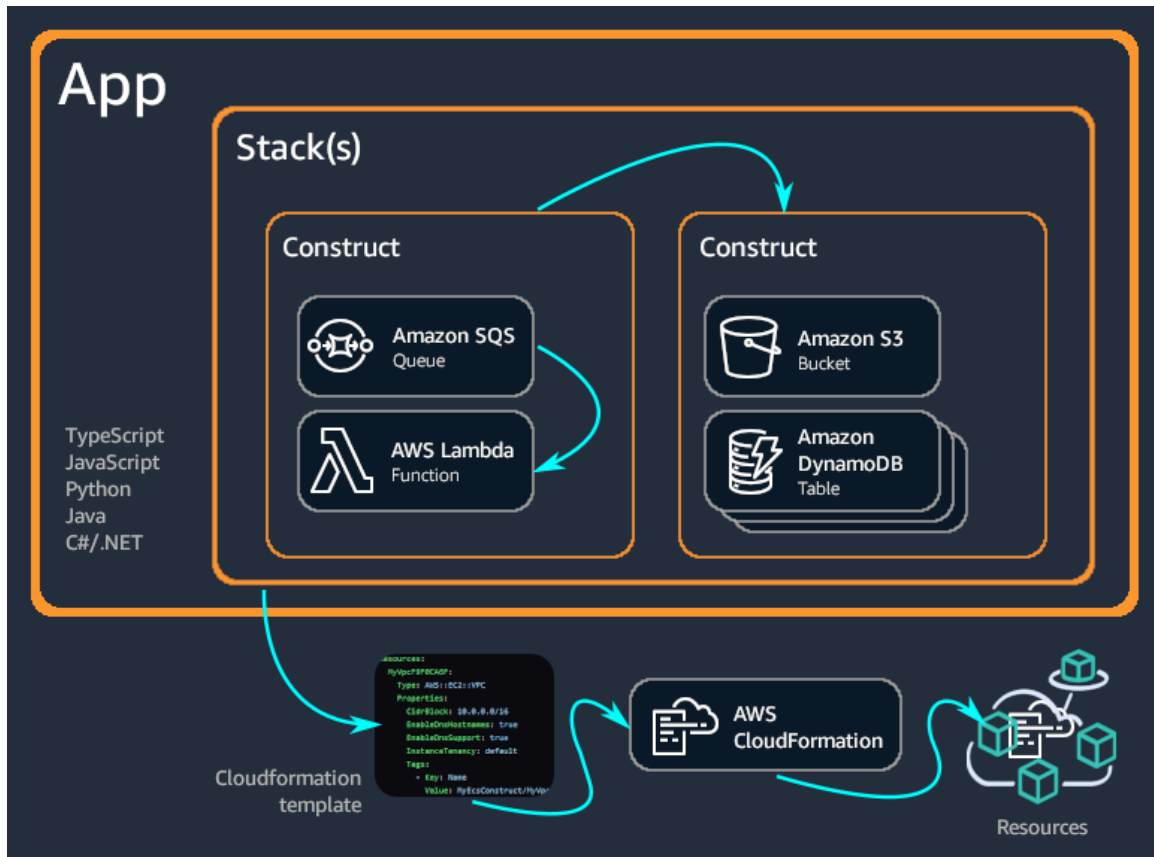


Figura 11.5. Esquema d'una aplicació d'AWS CDK
Font: docs.aws.amazon.com/cdk/v2 [33]

Com es pot veure en la figura 11.5 permet fer ús dels llenguatges de programació *TypeScript*, *JavaScript*, *Python*, *Java* i *C#/.NET* per poder desenvolupar la infraestructura creant l'aplicació d'AWS CDK. A més, com es pot veure en la mateixa figura una aplicació pot estar formada per un o més Stacks, que a la vegada estan formats per Constructs, que vindrien a ser els blocs bàsics de construcció. Aquests Constructs són components reutilitzables que estan formats per diferents recursos d'AWS relacionats entre si. Com també es pot veure en la figura els Constructs es poden traspasar informació entre ells, com també ho poden fer els Stacks.

Un cop es desenvolupa la infraestructura mitjançant codi i es fa el desplegament, s'obté la plantilla de CloudFormation. AWS CloudFormation [34] és un servei d'Amazon que ajuda a configurar els recursos d'AWS. Es fa ús de la plantilla, que descriu tots els recursos i les seves propietats, per crear un stack de CloudFormation. Un cop creat el stack, es posen en marxa tots els recursos definits en ell, tenint en compte les relacions que hi ha entre si.

En el nostre cas la nostra aplicació d'AWS CDK està formada per cinc stacks relacionats entre si i cada stack està format solament per un construct, que a la vegada està format per diversos recursos. En primer cas tindríem el stack principal, anomenat Analytics stack, que conté els recursos comuns i necessaris per als altres stacks. Com es pot veure en la figura 11.6 els recursos principals que es desenvolupen són el workgroup d'Athena i el database de Glue.

```

this.workgroup = new athena.CfnWorkGroup(this, 'WorkGroup', {
  name: 'analyticsWorkGroup',
  description: 'WorkGroup for Analytics',
  workGroupConfiguration: {
    resultConfiguration: {
      outputLocation: `s3://${this.resultBucket.bucketName}/`,
    },
  },
});

this.database = new glue.CfnDatabase(this, 'Database', {
  catalogId: this.account,
  databaseInput: {
    name: 'analytics-database',
    locationUri: `s3://${this.resultBucket.bucketName}/`,
    description: 'Database for Analytics',
  },
});

```

Figura 11.6. Recursos del stack d'Analítica

El workgroup d'Amazon Athena permet aïllar les consultes d'Athena que es realitzin en aquest workgroup d'altres workgroups dintre del mateix compte d'AWS. Per altra banda, la base de dades o database permet crear una agrupació de les taules del AWS Glue Data Catalog, ja que cada taula ha de pertànyer a un database.

Després del stack principal, es desenvolupen els tres stacks corresponents als tres tipus de dades que s'analitzen.

Com s'ha dit anteriorment respecte a les connexions les dades s'envien directament al seu bucket corresponent al S3, i per tant en el stack de les connexions solament tindrem el desenvolupament del bucket S3 i de la taula de Glue que serà utilitzada per Athena.

Mentre que pels playlogs i l'audit trail es desenvolupen més serveis per poder dur a terme l'escriptura i lectura d'aquestes dades. Al ser similars els dos stacks, agafarem com a exemple el stack de l'audit trail i d'aquesta manera mostrar com es desenvolupen els diferents serveis que es troben inclosos en el stack dels playlogs i en menor mesura en el stack de les connexions.

Com es pot veure en la figura 11.7, aquest stack obté per paràmetre el database del stack principal, ja que s'ha d'afegir la taula de Glue que desenvolupa en aquest stack. A continuació es crea un rol que és utilitzat pel recurs de Kinesis Data Firehose. L'objectiu d'aquest rol és poder donar-li els permisos necessaris per poder dur a terme de manera correcta les funcions de Kinesis Data Firehose. Després d'això també es desenvolupa i es configura el bucket de l'audit trail, i si li dona al rol de Firehose el permís d'escriptura en aquest bucket.

```

const database = props.database;

// IAM Roles

const firehoseRole = new iam.Role(this, 'Role', {
  assumedBy: new iam.ServicePrincipal('firehose.amazonaws.com'),
  roleName: 'firehose-audit-trail-role',
});

// S3 Buckets

const auditTrialBucket = new s3.Bucket(this, 'AuditTrialBucket', {
  bucketName: getBucketName('waapiti-audit-trail'),
  publicReadAccess: false,
  blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
  versioned: false,
  removalPolicy: RemovalPolicy.DESTROY,
});
auditTrialBucket.grantWrite(firehoseRole);

```

Figura 11.7. Part inicial del codi del stack de l'audit trail

Com es pot veure en la figura 11.8 es desenvolupa la taula d'AWS Glue que és utilitzada per Amazon Athena.

```

const auditTrailTable = new glue.CfnTable(this, 'AuditTrailTable', {
  databaseName: database.ref,
  catalogId: this.account,
  tableInput: {
    name: 'audit_trail',
    description: 'AuditTrail Table',
    storageDescriptor: {
      columns: [
        { name: 'username', type: 'string' },
        { name: 'usertype', type: 'string' },
        { name: 'timestamp', type: 'timestamp' },
        { name: 'entityId', type: 'int' },
        { name: 'entityType', type: 'string' },
        { name: 'action', type: 'string' },
      ],
      location: `s3://${auditTrialBucket.bucketName}/`,
      inputFormat: 'org.apache.hadoop.mapred.TextInputFormat',
      outputFormat: 'org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat',
      serdeInfo: {
        serializationLibrary: 'org.openx.data.jsonserde.JsonSerDe',
      },
      tableType: 'EXTERNAL_TABLE',
      partitionKeys: [
        { name: 'date', type: 'date' },
        { name: 'userid', type: 'int' },
        { name: 'clientid', type: 'int' },
      ],
    },
  },
});

```

Figura 11.8. Codi que implementa la taula de l'audit trail

Com es pot veure en la figura, s'han de definir les diferents columnes de la taula, que vindrien a ser les variables que presenten les dades. A més, indiquem on es troben les dades, de tal manera de què Amazon Athena pugui saber on buscar-les. També s'indica el format de les dades, que en el nostre cas, el format del document emmagatzemat en el bucket és un document JSON. Finalment, en la part de sota de la definició de la taula definim les diferents claus de partició.

Com s'ha dit anteriorment les particions fan que la lectura de les dades per part d'Amazon Athena sigui més eficient. L'ús de les particions fa que les consultes d'Amazon Athena no llegeixin totes les dades que hi ha en el bucket. Això s'aconsegueix indicant en la consulta quina partició es vol obtenir i, per tant, Amazon Athena solament buscarà dintre de les carpetes i subcarpetes iguals a aquesta partició. Les particions s'han creat tenint en compte els diferents casos d'ús i que d'aquesta manera es puguin aprofitar el màxim nombre de vegades. Respecte a l'audit trail hem fet que les particions siguin respecte a aquestes claus: date/userid/clientid com es pot veure en la figura 11.9. Mentre que pels playlogs i les connexions serien les següents: date/clientid. Per aquestes particions s'ha utilitzat l'estil Apache Hive, que consisteix en el fet que les rutes de les dades continguin els valors i les claus connectades pel signe igual, com per exemple podria ser en el cas de l'audit trail: **date=2024-04-30/userid=1/clientid=0/**. L'ús d'aquest estil ens permet crear noves particions mitjançant consultes d'Amazon Athena.

Partitions					
Filter partitions				< 1	
	date	userid	clientid	Files	Properties
☐	2024-05-07	1	509	View files	View Properties
☐	2024-05-07	1	1	View files	View Properties
☐	2024-04-30	1	0	View files	View Properties
☐	2024-05-07	1	454	View files	View Properties
☐	2024-05-07	1	0	View files	View Properties
☐	2024-05-07	1	23	View files	View Properties

Figura 11.9. Particions de la taula de Glue de l'audit trail
Font: Consola d'AWS

Un cop definida la taula d'AWS Glue, passem a implementar el stream de Kinesis Data Stream i a donar els permisos necessaris al rol creat anteriorment per tal que Kinesis Data Firehose pugui llegir del stream, com es pot veure en la figura 11.10.


```

const auditTrailStream = new kinesis.Stream(this, 'AuditTrailStream', {
  streamName: 'AuditTrailStream',
});

// Kinesis Data Firehose

const firehosePolicy = new iam.Policy(this, 'FirehoseAuditTrailPolicy', {
  roles: [firehoseRole],
  statements: [
    new iam.PolicyStatement({
      effect: iam.Effect.ALLOW,
      resources: [auditTrailStream.streamArn],
      actions: [
        'kinesis:DescribeStream',
        'kinesis:GetShardIterator',
        'kinesis:GetRecords',
        'kinesis:ListShards',
        'kinesis:ListStreams',
      ],
    }),
  ],
});

auditTrailStream.grantRead(firehoseRole);

```

Figura 11.10. Codi que implementa el stream de Kinesis Data Stream

Un cop implementat el stream i donats els permisos necessaris, passem a implementar el servei de Kinesis Data Stream i a unir-lo amb el rol creat anteriorment, com es pot veure en les figures 11.11 i 11.12.

```

const auditTrailFirehose = new firehose.CfnDeliveryStream(
  this,
  'AuditTrailFirehose',
  {
    deliveryStreamType: 'KinesisStreamAsSource',
    kinesisStreamSourceConfiguration: {
      kinesisStreamArn: auditTrailStream.streamArn,
      roleArn: firehoseRole.roleArn,
    },
    deliveryStreamName: 'AuditTrailFirehose',
    extendedS3DestinationConfiguration: {
      bucketArn: auditTrailBucket.bucketArn,
      roleArn: firehoseRole.roleArn,
      prefix:
        'date={!{partitionKeyFromQuery:date}/userid={!{partitionKeyFromQuery:userid}/clientid={!{partitionKeyFromQuery:clientid}/',
      bufferingHints: {
        intervalInSeconds: 60,
      },
      dynamicPartitioningConfiguration: {
        enabled: true,
      },
      processingConfiguration: {
        enabled: true,
        processors: [
          {
            type: 'AppendDelimiterToRecord',

```

Figura 11.11. Codi de la implementació de Kinesis Data Firehose

```

parameters: [
  {
    parameterName: 'Delimiter',
    parameterValue: '\\n',
  },
],
},
{
  type: 'MetadataExtraction',
  parameters: [
    {
      parameterName: 'MetadataExtractionQuery',
      parameterValue:
        '{date: .date, userid: .userid, clientid: .clientid}',
    },
    {
      parameterName: 'JsonParsingEngine',
      parameterValue: 'JQ-1.6',
    },
    {
      parameterName: 'RoleArn',
      parameterValue: firehoseRole.roleArn,
    },
  ],
},
],
errorOutputPrefix: 'errors/!{firehose:error-output-type}/',
}
);

auditTrailFirehose.addDependency(
  firehosePolicy.node.defaultChild as CfnResource

```

Figura 11.12. Codi de la implementació de Kinesis Data Firehose

Com es pot veure en aquestes dues figures, en la configuració del servei de Kinesis Data Firehose indiquem quin és el stream d'on s'obtidran les dades i quin és rol que hi participa. A més, també indiquem el bucket de S3 on s'emmagatzemaran i la forma en què realitzarà la partició dinàmica. Com hem dit abans la partició es realitza fent és de l'estil Apache Hive, com es pot veure en el codi. També es configura l'interval de temps en què farà cada càrrega de dades en S3, que en el nostre cas és de 60 segons. Com s'ha dit anteriorment, les dades s'escriuran en documents JSON i cada dada estarà separada per un salt de línia, per tal que Amazon Athena les pugui llegir correctament.

A part dels stacks explicats fins ara tenim el stack del servei d'analítica, que ens permet fer el desplegament en el núvol del mòdul analític. Per poder realitzar aquest desplegament s'ha fet ús de diversos serveis d'Amazon. L'ús d'aquests serveis permet que un cop el mòdul estigui en línia pugui funcionar correctament i que sigui altament escalable.

Un cop tenim tots els stacks desenvolupats procedim a unir-los per crear l'aplicació, com es pot veure en la figura 11.13.

```

const app = new cdk.App();

/** Base analytics */
const analytics = new CdkAnalyticsStack(app, 'AnalyticsBaseStack', {
  env: { account: process.env.ACCOUNT!, region: 'eu-west-1' },
});

/** Audit trail */
new CdkAuditTrailStack(app, 'AnalyticsAuditTrailStack', {
  env: { account: process.env.ACCOUNT!, region: 'eu-west-1' },
  database: analytics.database,
});

/** Connections */
const connection = new CdkConnectionStack(app, 'CdkConnectionStack', {
  env: { account: process.env.ACCOUNT!, region: 'eu-west-1' },
  database: analytics.database,
});

/** Playlog */
new CdkPlaylogStack(app, 'AnalyticsPlaylogStack', {
  env: { account: process.env.ACCOUNT!, region: 'eu-west-1' },
  database: analytics.database,
});

/** Service */
new CdkAnalysisServiceStack(app, 'AnalyticsServiceStack', {
  env: { account: process.env.ACCOUNT!, region: 'eu-west-1' },
  connectionBucket: connection.connectionBucket,
  resultBucket: analytics.resultBucket,
  workgroup: analytics.workgroup,
  database: analytics.database,
});

```

Figura 11.13. Codi de l'aplicació del CDK d'Analítica

Com es pot veure en la figura 11.13 primer creem l'aplicació i després anem afegint els diferents stacks. Primer de tot afegim el stack d'Analítica, ja que és necessari perquè els altres tres stacks fan ús del database creat en aquest stack. Després d'això afegim els stacks corresponents als tres tipus de dades i finalment afegim el stack del servei.

Un cop implementada l'aplicació del CDK realitzem el seu desplegament. Com indica la figura 11.5, al realitzar el desplegament obtenim les plantilles dels diferents stacks, com es pot veure com a exemple la plantilla del stack de l'audit trail en la figura 11.14.

```

{
  "Resources": {
    "Role1ABCC5F0": {
      "Type": "AWS::IAM::Role",
      "Properties": {
        "AssumeRolePolicyDocument": {
          "Statement": [
            {
              "Action": "sts:AssumeRole",
              "Effect": "Allow",
              "Principal": {
                "Service": "firehose.amazonaws.com"
              }
            }
          ]
        },
        "Version": "2012-10-17"
      },
      "RoleName": "firehose-audit-trail-role"
    },
    "Metadata": {
      "aws:cdk:path": "CdkAuditTrailStack/Role/Resource"
    }
  },
  "RoleDefaultPolicy5FFB7DAB": {
    "Type": "AWS::IAM::Policy",
    "Properties": {
      "PolicyDocument": {
        "Statement": [
          {
            "Action": [
              "s3:Abort*",
              "s3:DeleteObject*",
              "s3:PutObject",
              "s3:PutObjectLegalHold",

```

```

      "AuditTrailTable": {
        "Type": "AWS::Glue::Table",
        "Properties": {
          "CatalogId": {
            "Ref": "AWS::AccountId"
          },
          "DatabaseName": {
            "Fn::ImportValue": "CdkAnalyticsStack:ExportsOutputRefDatabase8534DE71"
          },
          "TableInput": {
            "Description": "AuditTrail Table",
            "Name": "audit_trail",
            "PartitionKeys": [
              {
                "Name": "date",
                "Type": "date"
              },
              {
                "Name": "userid",
                "Type": "int"
              },
              {
                "Name": "clientid",
                "Type": "int"
              }
            ],
            "StorageDescriptor": {
              "Columns": [
                {
                  "Name": "timestamp",
                  "Type": "timestamp"
                }
              ]
            }
          }
        }
      },
      "StorageDescriptor": {
        "Columns": [
          {
            "Name": "timestamp",
            "Type": "timestamp"
          }
        ]
      }
    }
  }
}

```

Figura 11.14. Plantilla del CloudFormation de l'audit trail
Font: Consola d'AWS

Com es pot veure en aquesta figura es troben identificats tots els recursos implementats per codi en els AWS CDK. Com s'ha dit anteriorment, el servei de CloudFormation posa en marxa tots els recursos descrits en la plantilla del stack com es pot veure en la figura 11.15.

Logical ID ▲	Physical ID ▼	Type ▼	Status ▼
Role	-	-	✔ CREATE_COMPLETE
AuditTrailBucket	-	-	✔ CREATE_COMPLETE
AuditTrailStream	-	-	✔ CREATE_COMPLETE
FirehoseAuditTrailPolicy	-	-	✔ CREATE_COMPLETE
AuditTrailFirehose	-	-	✔ CREATE_COMPLETE
CDKMetadata	-	-	✔ CREATE_COMPLETE
AuditTrailTable	audit_trail	AWS::Glue::Table	✔ CREATE_COMPLETE

Figura 11.15. Recursos de l'audit trail
Font: Consola d'AWS

Un cop executada la plantilla, ja podem comprovar a través de la consola la formació de tots els recursos de l'aplicació. Com es pot veure en la figura 11.16, en el cas d'Amazon Athena, ja s'han creat les diferents taules amb les seves columnes i particions.

▼ Tables (3) < 1 >	
audit_trail	Partitioned ⋮
timestamp	timestamp ⋮
entityid	int ⋮
entitytype	string ⋮
action	string ⋮
data	string ⋮
date	date (Partitioned) ⋮
userid	int (Partitioned) ⋮
clientid	int (Partitioned) ⋮
connection	Partitioned ⋮
playlog	Partitioned ⋮
► Views (0) < 1 >	

Figura 11.16. Taules d'Amazon Athena
Font: Consola d'AWS

11.2. Backend

En aquest subapartat passarem la implementació dels diferents microserveis o mòduls que componen el projecte. Abans de passar a parlar explicarem els punts principals del framework de *NestJS*, que és el que s'ha utilitzat per desenvolupar el backend.

NestJS és un framework que permet la creació d'una API REST fent ús de la combinació d'elements d'OOP (*Object Oriented Programming*), FP (*Functional Programming*) i FRP (*Functional Reactive Programming*). L'estructura d'aquest framework es basa en tres elements: mòduls, controladors i proveïdors.

Els mòduls són les classes de NestJS que permeten organitzar l'aplicació. Els mòduls són importats per part del mòdul principal, anomenat *Application Module*, per tal que puguin ser utilitzats. A més, els mòduls poden ser importats per altres mòduls a part del principal per tal de poder utilitzar les seves funcionalitats dintre d'aquest mòdul. En la figura 11.17 es pot veure un exemple de l'estructura dels mòduls.

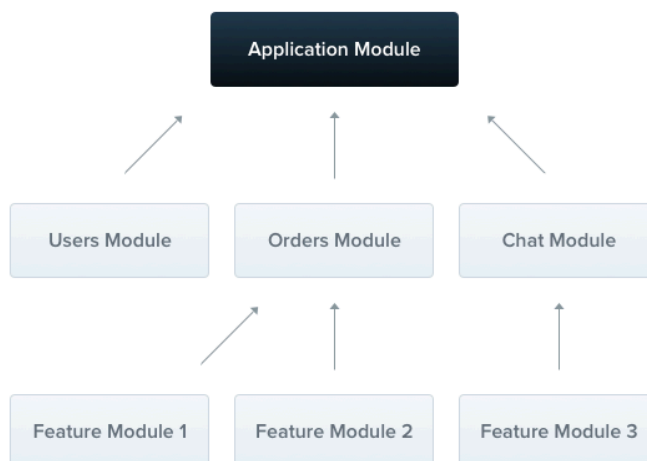


Figura 11.17. Estructura dels mòduls en NestJS

Font: NestJS [2]

Els controladors en *NestJS* són els que s'encarreguen de rebre les peticions HTTP i processar-les. Aquestes peticions HTTP són enviades als seus proveïdors corresponents.

En la figura 11.18 es pot veure un exemple de l'estructura dels controladors.

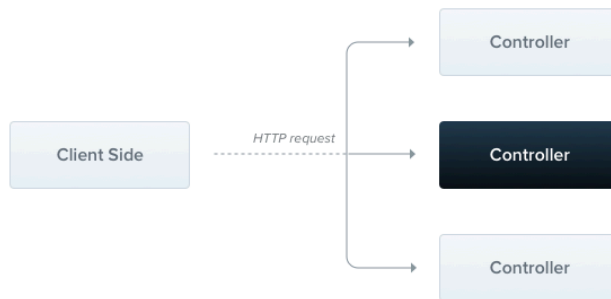


Figura 11.18. Estructura dels controladors en NestJS
Font: NestJS [2]

Els proveïdors són classes de *NestJS* que poden actuar com serveis, factories, helpers, repositoris, entre altres. Els proveïdors estan dissenyats per poder injectats com a dependències en mòduls o altres components.

En el cas d'aquest projecte s'han utilitzat els proveïdors com a serveis, és a dir, com a classes que contenen la lògica de negoci de l'aplicació. Per tal de poder fer ús d'aquests serveis, es fa la seva importació en els seus mòduls corresponents. En la figura 11.19 es pot veure un exemple de l'estructura dels proveïdors.

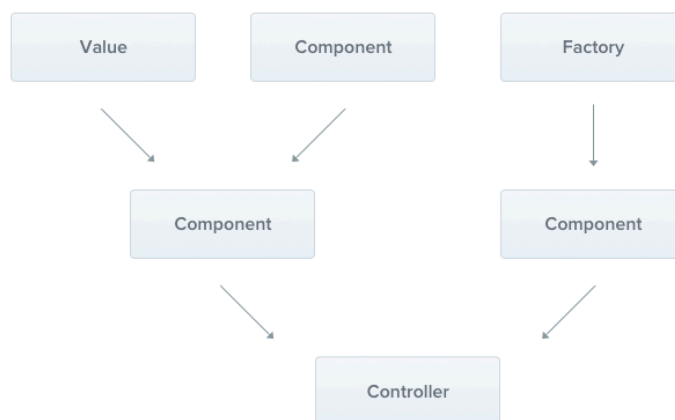


Figura 11.19. Estructura dels proveïdors en NestJS
Font: NestJS [2]

Com s'havia dit anteriorment en l'apartat 9, la plataforma està formada per diferents microserveis que es comuniquen entre si mitjançant TCP. En cadascun d'aquests microserveis hi haurà un mòdul principal per aquell microservei. A continuació passarem a explicar en detall la implementació de cada microservei que ha intervingut en aquest projecte.

11.2.1. Mòdul analític

Com s'ha dit anteriorment aquest microservei, és el microservei que s'ha desenvolupat en la seva totalitat i és qui s'encarrega de dur a terme les funcions d'obtenció de dades dels serveis d'AWS. Encara que també té altres funcionalitats menors.

Com es pot veure en la figura 11.20 aquest microservei està format per un conjunt de classes relacionades amb els tipus de dades, les classes cron, que s'explicaran més endavant, i els serveis que són utilitzats per les altres classes.

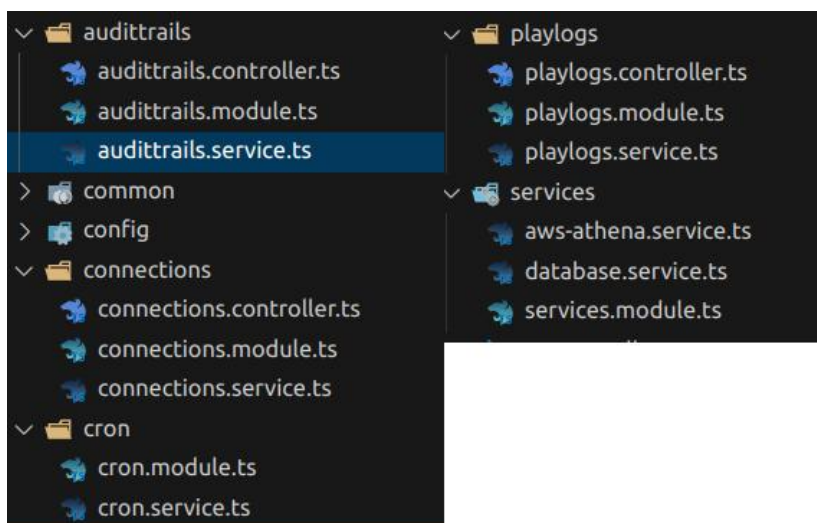


Figura 11.20. Estructura del mòdul analític

Abans d'explicar les classes relacionades amb les dades i el cron, es farà una explicació dels serveis utilitzats per aquests.

El servei d'AWS Athena, és el servei que té com a funció realitzar les consultes al servei d'Amazon Athena. Per tal de poder realitzar les consultes s'ha d'importar la llibreria d'Athena, que té les funcions necessàries per poder crear les funcions que realitzin la consulta. A més, per poder obtenir les respostes a les consultes realitzades s'ha de configurar el servei perquè realitzi les consultes a la instància d'Athena que havíem creat prèviament, com es pot veure en la figura 11.21. La funció pública que hi ha en aquesta figura s'encarrega de retornar la consulta en forma d'arrays d'objectes.

```

async query(sqlQuery: string): Promise<any> {
  const queryExecutionInput = {
    QueryString: sqlQuery,
    QueryExecutionContext: {
      Database: this.database,
      Catalog: this.catalog,
    },
    WorkGroup: this.workGroup,
    ResultReuseConfiguration: this.resultReuseConfiguration,
  };

  const { QueryExecutionId } = await this.client.send(
    new StartQueryExecutionCommand(queryExecutionInput),
  );

  const response =
    await this.checkQueryExeexutionStateAndGetData(QueryExecutionId);
  return response;
}

```

Figura 11.21. Funció pública del servei d'Athena

La funció *checkQueryExeexutionStateAndGetData* s'encarrega d'anar comprovant cada 1000 segons, com es pot veure en la figura 11.22, l'execució de la consulta, ja que les consultes d'Athena poden tenir un cert endarreriment. D'aquesta manera les consultes no es perden.

```

if (
  state === QueryExecutionState.QUEUED ||
  state === QueryExecutionState.RUNNING
) {
  await this.timeout(1000);
  return await this.checkQueryExeexutionStateAndGetData[QueryExecutionId];
} else if (state === QueryExecutionState.SUCCEEDED) {
  return await this.getQueryResults(QueryExecutionId);
} else if (state === QueryExecutionState.FAILED) {
  throw new Error(
    `Query failed: ${response.QueryExecution.Status.StateChangeReason}`,
  );
} else if (state === QueryExecutionState.CANCELLED) {
  throw new Error(`Query was cancelled`);
}
}

```

Figura 11.22. Tros de codi de la funció *checkQueryExeexutionStateAndGetData*

Un cop l'estat de la consulta és exitós es crida la funció que s'encarrega d'obtenir els resultats de la consulta. Dintre d'aquesta funció processarem els resultats de la consulta per tal que cada columna de la taula d'Amazon Athena sigui una variable dels objectes que es retornen en forma d'array.

Per altra banda, també tenim el servei de base de dades, que s'encarrega de donar la funcionalitat de poder connectar i realitzar consultes a la base de dades de la plataforma.

Com es pot veure en la figura 11.23, el mòdul de serveis té com a proveïdors els serveis esmentats fins ara. També podem veure que aquests serveis s'exporten i d'aquesta manera es podrà fer ús de les funcionalitats dels serveis en les classes on s'importin, que tractaran de les classes relacionades amb els tipus de dades.


```
@Module({
  providers: [AWSAthenaService, DatabaseService],
  exports: [AWSAthenaService, DatabaseService],
})
export class ServicesModule {}
```

Figura 11.23. Mòdul dels serveis

A continuació es farà una explicació del conjunt de classes principals d'aquest microservei, que vindrien a ser aquelles que estan relacionades amb els tipus de dades que analitzem. Per poder entendre en profunditat com funcionen aquest conjunt de classes ens centrarem en un sol tipus, ja que l'estructura és semblant entre els tres tipus de dades. Encara que també es farà una explicació de totes les funcions de manera individual.

Com a exemple d'aquest conjunt de classes ens centrarem en les connexions. Com es pot veure en la figura 11.24 tenim el mòdul indicat mitjançant el decorador `@Module`. En aquest mòdul podem veure que es fa una importació del mòdul dels serveis per poder utilitzar les funcions de base de dades i d'Amazon Athena. També es defineix el controlador de les connexions i es posa com a proveïdor el servei de les connexions. A més, es realitza una exportació d'aquest mòdul per tal de poder-lo utilitzar en el mòdul principal.

```
@Module({
  imports: [ServicesModule],
  controllers: [ConnectionsController],
  providers: [ConnectionsService],
})
export class ConnectionsModule {}
```

Figura 11.24. Mòdul de Connexions

Com es pot veure en la figura 11.25, el controlador de les connexions té com a funció definir les rutes d'accés a les que criada el microservei Api Gateway i executar les funcions del servei de connexions.

```
@Controller('connection')
export class ConnectionsController {
  constructor(private readonly connections: ConnectionsService) {}

  @MessagePattern({ cmd: 'connections.get' })
  async getConnections(data): Promise<CoreResponse> {
    return this.connections.getConnections(data.deviceId, data.user);
  }

  @MessagePattern({ cmd: 'connections.getByDate' })
  async getConnectionsByDate(data): Promise<CoreResponse> {
    return this.connections.getConnectionsByDate(
      data.since,
      data.until,
      data.idClient,
      data.idLocationGroup,
      data.user,
    );
  }
}
```

Figura 11.25. Controlador de les connexions

El servei de connexions és qui s'encarrega de realitzar tota la lògica de negoci relacionada amb la lectura de les connexions. En la figura 11.26 es pot veure el constructor de la classe i el decorador `@Injectable`, que indica que la classe actuarà com un proveïdor.

```
@Injectable()
export class ConnectionsService {
  constructor() {
    private readonly athena: AWSAthenaService,
    private readonly database: DatabaseService,
    private readonly guard: GuardService,
  } {}
}
```

Figura 11.26. Servei de les connexions

Pel que fa a la lògica de negoci dels endpoints de les classes relacionades amb les dades és molt similar entre si. Per tant, s'agafarà com a exemple el endpoint de `connections/:deviceId`.

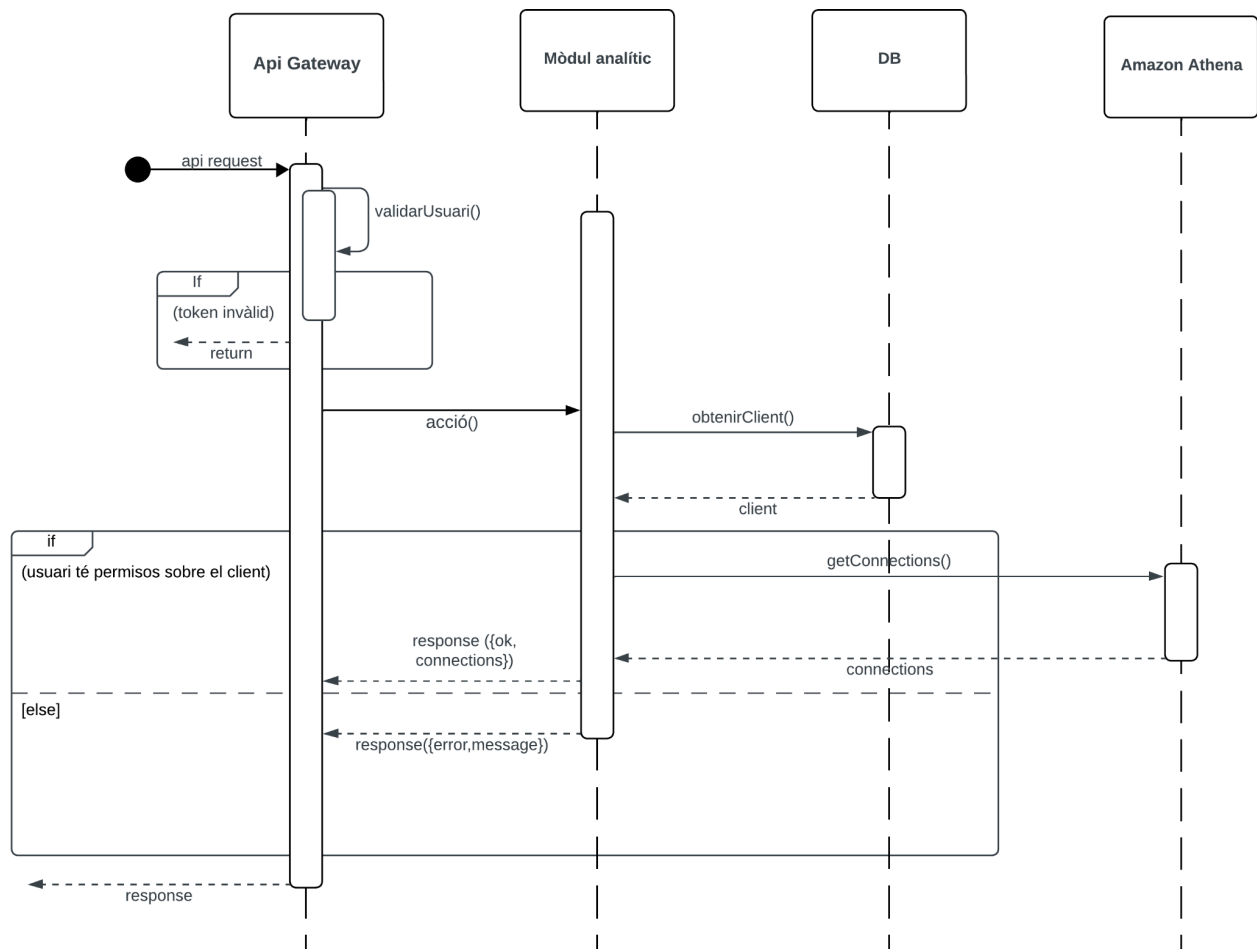


Figura 11.27. Esquema de seqüència del endpoint `connections/:deviceId`

En la figura 11.27 es pot veure l'esquema de seqüència de l'endpoint *connections/:deviceId* i que es pot generalitzar als altres endpoints relacionats amb els altres tipus de dades. Com es pot veure, l'Api Gateway rep la petició i la valida. Si és la petició és vàlida s'envia al mòdul analític. En aquest mòdul s'obté el client des de la base de dades i es comprova que l'usuari té els permisos suficients per obtenir les dades d'aquest client. Si l'usuari té els permisos suficients es realitza la consulta a la taula de connexions d'Amazon Athena i s'obtenen les 20 últimes connexions del player indicat. Un cop Amazon Athena ens retorna una resposta, el mòdul analític la retorna a l'Api Gateway i aquesta s'encarrega de retornar-la a l'usuari.

El primer pas per poder obtenir les dades de les connexions sobre un player en concret és definir la ruta d'accés en el servei d'Api Gateway. En la figura 11.28 es pot veure el codi que defineix aquesta ruta.

```
/**
 * Get 20 last connctions of a device
 * [GET] /connections/:deviceId
 */
@Get('/:deviceId')
@ApiBearerAuth("User Token")
@UseGuards(JwtGuard, UserGuard, PolicyGuard)
async getPlaylogs(
  @Req() req: AuthRequest,
  @Res() res: Response,
  @Param('deviceId', ParseIntPipe) deviceId: number
): Promise<void> {
  const response = await this.analyticalService.send<CoreResponse>('connections.get', { deviceId, user: req.user });
  res.status(response.httpStatus).json(response);
}
```

Figura 11.28. Ruta d'accés al endpoint per obtenir 20 últimes connexions en l'Api Gateway

En el codi es pot observar de què fem ús del decorador *@UseGuards*, aquest decorador indica de què s'han de fer unes funcions de validació abans de poder accedir a la funció d'obtenció de les connexions. Els altres decoradors permeten definir la ruta d'accés i garantir la funcionalitat completa de TypeScript. Un cop processades les dades de la petició, s'envia l'acció al mòdul analític per processar-la.

En el mòdul analític s'accedeix a la funció *getConnections()* del servei de les connexions, que es pot observar en la figura 11.29. En aquesta funció, primer realitzem una consulta SQL al servei de base de dades i d'aquesta manera obtenim el client al qui li pertany el player del qual volem saber les seves connexions. Un cop obtingut el client comprovem que l'usuari que vol fer la petició té permisos el client obtingut. Si no en té es retorna l'error, però si en té, es realitza la consulta SQL al servei d'Amazon Athena. Aquesta consulta aprofita l'ús de les particions en Athena i solament es busca dintre de les particions on el *clientId* sigui l'indicat en la consulta. On cop obtinguda la resposta d'Amazon Athena es retorna a l'Api Gateway.

```

async getConnections(
  deviceId: number,
  user: UserCache,
): Promise<CoreResponse> {
  try {
    const clientId = await this.database
      .query(
        `SELECT l.clientid FROM device d LEFT JOIN location l ON d.locationId = l.id WHERE d.id = ${deviceId}`,
      )
      .then((res) => res[0].clientId);
    const guardData = await this.guard.clientCheckOwn(clientId, user);
    if (!guardData)
      throw new Error('User does not have permissions over the client');
    const sqlQuery =
      'SELECT * FROM connection WHERE clientid = ' +
      clientId +
      ' AND playerid = ' +
      deviceId +
      ' LIMIT 20';
    console.log(sqlQuery);

    const response = await this.athena.query(sqlQuery);
    return {
      httpStatus: HttpStatus.OK,
      connections: response,
    };
  } catch (error) {
    throw error;
  }
}

```

Figura 11.29. Funció encarregada d'obtenir les 20 últimes connexions d'un player

En la següent taula 11.1 es poden veure els detalls de la crida a l'API que implementa aquesta funcionalitat.

Descripció	Obtenir últimes 20 connexions d'un player
Mètode	GET
URL	connections/:deviceId
Paràmetres	deviceId: Number
Body	-
Resposta	200: { connections: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Taula 11.1. API: Obtenir últimes 20 connexions
Font: Elaboració pròpia

Pel que fa a les altres funcions de les classes *Connections*, *Audit Trail* i *Playlog* segueixen una estructura semblant, com la que es pot veure en la figura 11.30. La causa de la seva semblança és del fet que aquest microservei té com a objectiu principal la lectura de les dades de diferents maneres.

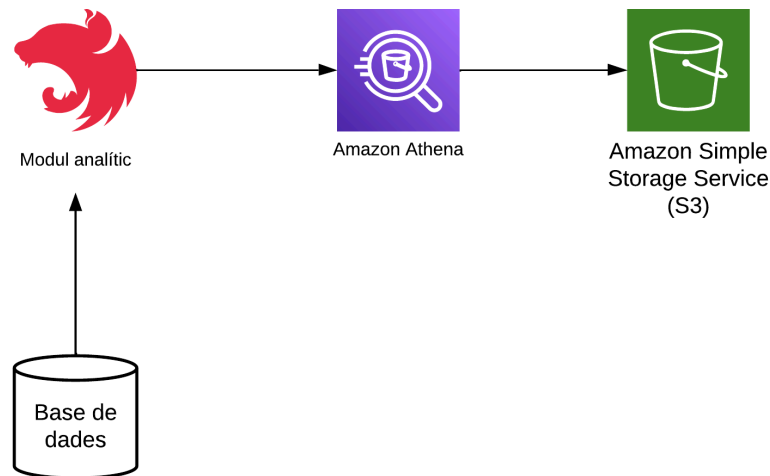


Figura 11.30. Estructura de les funcions de lectura de les dades
Font: Elaboració pròpia

A continuació farem una breu introducció a aquestes funcions i explicarem en detall les especificitats de cada funció.

Pel que fa a les connexions també tenim la funció que ens permet obtenir les connexions entre dues dates d'un client o una location group, com es pot veure en la taula 11.2. Al fer la consulta aprofitem les dates per solament buscar dintre de les particions entre aquestes dates i d'aquesta manera millorem l'eficiència de la consulta. El resultat de la consulta dependrà dels diferents permisos que tingui l'usuari, com es pot veure en el tros de codi de la figura 11.31, on l'usuari pot ser un location group o un administrador, entre altres.

```

} else if (guardData.locationGroupId) {
  //LocationGroup
  const locationGroupDBclientId = await this.database.query(
    `SELECT clientid FROM location_group WHERE id = ${guardData.locationGroupId}`,
  );
  const locationIds = await this.database
    .query(
      `SELECT locationId FROM location_group_assn WHERE locationGroupId = ${guardData.locationGroupId}`,
    )
    .then((res) => res.map((location) => location.locationId));
  sqlQuery = `SELECT * FROM connection c
    timestamp BETWEEN TIMESTAMP '${start}' AND TIMESTAMP '${end}'`;
} else {
  //Admin
  sqlQuery = `SELECT * FROM connection
    WHERE
    date BETWEEN DATE('${sinceDate}') AND DATE('${untilDate}')
    AND
    timestamp BETWEEN TIMESTAMP '${start}' AND TIMESTAMP '${end}'`;
}

```

Figura 11.31. Tros de codi de la funció getConnectionByDate

Descripció	Obtenir connexions entre dues dates
Mètode	GET
URL	connections/date
Paràmetres	-
Body	since: string until: string clientId: Number LocationGroupId: Number
Resposta	200: { connections: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Taula 11.2. API: Obtenir connexions

Font: Elaboració pròpia

Pel que fa a la classe dels playlogs en aquest mòdul tenim la funció que té com a objectiu obtenir els playlogs d'un player específic entre dues dates. En la taula 11.3 es pot veure en més detall. Com totes les funcions que tracten de realitzar consultes a Amazon Athena, sempre fan ús de les particions en les consultes per fer-la més eficient.

Descripció	Obtenir playlogs entre dues dates
Mètode	GET
URL	playlogs/:deviceId/date
Paràmetres	deviceId: Number
Body	since: string until: string
Resposta	200: { playlogs: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Taula 11.3. API: Obtenir playlogs

Font: Elaboració pròpia

Pel que fa a la classe d'audit trail tindríem tres funcions principals que tenen com a objectiu obtenir les accions en plataforma d'una entitat en específic de la plataforma, d'un client o d'un usuari, com es pot veure en les tres taules següents. A més, per millorar la seguretat i donar privacitat als usuaris administradors s'ha eliminat en la resposta el nom de l'usuari i el seu identificador sense afectar els documents emmagatzemats en S3, com es pot veure en la

figura 11.32. D'aquesta manera els usuaris que no siguin administradors no podran veure el nom de les accions produïdes pels administradors, solament podran veure que l'han realitzat usuaris administradors.

```
if (user.type !== 'admin') {
  response.forEach((element) => {
    if (element.usertype === 'admin') {
      element.username = 'Admin';
      element.userid = null;
    }
  });
}
```

Figura 11.32. Codi que aporta privacitat a la resposta

En la figura 11.33 es pot observar la part visual de la plataforma, desenvolupada pel desenvolupador frontend de l'empresa, on es mostren les dades retornades per la funció de la taula 11.5.

The screenshot shows a web application interface for an 'Audit Trail'. On the left, there are several filter controls: 'Filter by type' (set to 'Client'), 'Filter by client' (set to 'Andorra Kebab'), 'Filter by entity type' (set to 'Calendar'), 'Since' (set to '06/02/2024'), and 'Until' (set to '06/08/2024'). The main area displays a table with the following data:

ACTION	TRIGGERED BY	ENTITY TYPE	DATE
ADD_SCHEDULE	daniel.andorra	Calendar	June 6, 2024 3:51:38 PM
DELETE_SCHEDULE	daniel.andorra	Calendar	June 6, 2024 3:52:00 PM
DELETE_SCHEDULE	daniel.andorra	Calendar	June 6, 2024 3:52:10 PM
EDIT_SCHEDULE	daniel.andorra	Calendar	June 6, 2024 3:52:18 PM
SAVE	daniel.andorra	Calendar	June 6, 2024 3:52:25 PM

The interface also includes a top navigation bar with a 'Demo User Admin' profile and a language selector set to 'EN'.

Figura 11.33. Captura de pantalla del retorn dels audit trails

Descripció	Obtenir audit trail entre dues dates d'una entitat
Mètode	GET
URL	audit_trail/entity
Paràmetres	-
Body	since: string until: string entityId: Number entityType: String
Resposta	200: { audit: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Taula 11.4. API: Obtenir audit trail d'una entitat
Font: Elaboració pròpia

Descripció	Obtenir audit trail entre dues dates d'un client
Mètode	GET
URL	audit_trail/client
Paràmetres	-
Body	since: string until: string clientId: Number
Resposta	200: { audit: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

Taula 11.5. API: Obtenir audit trail d'un client
Font: Elaboració pròpia

Descripció	Obtenir audit trail entre dues dates d'un usuari
Mètode	GET
URL	audit_trail/user
Paràmetres	-
Body	since: string until: string userId: Number
Resposta	200: { audit: Object; httpStatus: Number; } 401: Unauthorized 404: Not Found

*Taula 11.6. API: Obtenir audit trail d'un usuari
Font: Elaboració pròpia*

A part de les classes de lectura relacionades amb les dades també la classe cron. Aquesta classe té funcions que estan programades per tal que es realitzin de manera periòdica. Aquesta funcionalitat ve donada pel paquet de NestJS `@nestjs/schedule`. Important aquest paquet, podem fer ús del decorador `@Cron`, que permet que cada cert temps s'executi la funció on està declarat. Per poder indicar cada quant s'executarà la funció, ho indiquem dintre del decorador `@Cron`, com es pot veure en la figura 11.34. En el nostre cas realitzem la tasca el primer minut de cada hora utilitzant aquest decorador: `@Cron('1 * * * *')`.

```
* * * * *
| | | | |
| | | | day of week
| | | | months
| | | day of month
| | hours
| minutes
seconds (optional)
```

*Figura 11.34. Indicadors de l'interval de temps en el cron
Font: NestJS [2]*

La funció d'aquesta tasca és obtenir les connexions de l'última hora dels players de la base de dades, es preparen i s'envien al S3. A més, per poder obtenir les dades mitjançant Amazon Athena es realitza una partició, si no existeix, en AWS Glue mitjançant la consulta que apareix en la figura 11.35. Finalment, un cop emmagatzemades les connexions en S3 eliminem de la base de dades les connexions a l'hora anterior. D'aquesta manera solament mantenim en la base dades l'última hora.

```

if (!devicesByDateAndClient[date][clientId]) {
  devicesByDateAndClient[date][clientId] = [];
  var sqlQuery = `ALTER TABLE connection ADD IF NOT EXISTS PARTITION (date = '${date}', clientId = ${clientId})`;
  await this.athena.query(sqlQuery);
}

```

Figura 11.35. Addició d'una partició a la taula d'AWS Glue

11.2.2. Core i Web Socket

En aquest subapartat es tractarà sobre les funcionalitats que s'han desenvolupat o adaptat dintre dels microserveis Core i Web Socket. Les funcionalitats desenvolupades en aquests dos microserveis és semblant. Pel Core, al ser el microservei central de la plataforma té les funcionalitats principals d'aquesta i, per tant, conté totes les accions que pot realitzar l'usuari. Per altra banda, el microservei de Web Socket s'encarrega de comunicar els players amb la plataforma i, per tant, té accés al contingut que reproduïxen, és a dir, els playlogs.

Per aquests microserveis s'ha desenvolupat un servei que s'encarrega d'enviar les dades al stream de Kinesis Data Stream corresponent, que pot ser el de l'audit trail o dels playlogs. Al ser tan semblants ens centrarem en el microservei Core per explicar els detalls de les funcionalitats.

En el microservei Core s'ha desenvolupat el servei de manera que pugui ser accessible per totes les classes del microservei que s'encarreguen de realitzar les accions en la plataforma. Com s'havia dit anteriorment, això s'aconsegueix important el mòdul del servei al mòdul de la classe.

L'esquema de connexions que segueix la funcionalitat de l'escriptura de l'audit trail es pot observar en la figura 11.36. Com es pot observar, abans d'enviar les dades al stream s'han de crear les particions mitjançant una consulta a Amazon Athena. A causa d'això en el servei de Kinesis també s'ha d'importar la llibreria d'Athena a part, de la de Kinesis. En el servei de Kinesis primer configurem tant Athena com Kinesis Data Stream i després afegim les dades en un format JSON. Com es pot veure en la figura 11.37, les dades s'envien al stream indicat i passats 60 segons o quan el stream estigui ple Kinesis Data Firehose s'encarrega d'escriure les dades en les subcarpetes que els hi correspongui depenent de les particions.

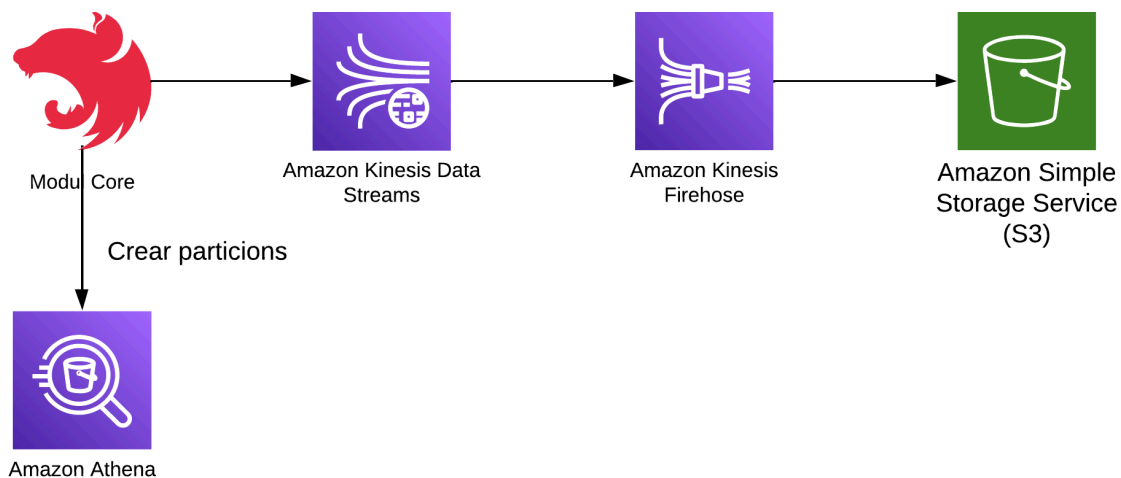


Figura 11.36. Esquema de les connexions de l'audit trail
Font: Elaboració pròpia

```

private async putRecord(data: any) {
  const params = {
    Data: Buffer.from(JSON.stringify(data)),
    PartitionKey: "1",
    StreamName: process.env.AWS_KINESIS_STREAM_NAME,
  };
  const command = new PutRecordCommand(params);
  return await this.kinesisClient.send(command);
}

```

Figura 11.37. Funció encarregada d'enviar les dades

Per poder tenir l'audit trail de totes les accions en plataforma realitzem la crida de la funció principal del servei de Kinesis en totes les funcions que creen algun canvi en la plataforma, com pot ser crear, editar, eliminar o publicar alguna entitat en la plataforma. En la figura 11.38 es pot veure un exemple en la funció que crea un player.

```

//Kinesis
const playerData = JSON.stringify(player)
this.kinesis.executeQueryAndPutResultsInKinesis(player.id, 'player', 'create', playerData, user, location.client.id);

```

Figura 11.38. Crida de la funció principal de Kinesis

Com es pot veure en aquesta figura li passem com a paràmetres les dades del player, l'acció que s'ha realitzat, quin usuari l'ha realitzat i a quin client pertany aquesta entitat.

12. Tests

En aquest apartat s'especifiquen els tests que s'han realitzat durant el desenvolupament del projecte per poder comprovar el bon funcionament del sistema.

Els tests que s'han realitzat han sigut els tests sobre la infraestructura i els tests d'integració. Respecte als tests d'infraestructura s'han dut a terme proves de manera directa sobre la infraestructura per poder comprovar el seu bon funcionament. Pel que fa als tests d'integració, s'han realitzat proves per comprovar que el sistema funciona de manera correcta de manera conjunta.

En aquest projecte no s'han realitzat tests unitaris, ja que no formen part del procés de desenvolupament de l'empresa. Això és degut al fet que els tests d'integració han sigut suficients per poder comprovar el bon funcionament de la plataforma. Encara que en un futur seria adient afegir en el procés de desenvolupament els tests unitaris per tal d'evitar la introducció d'errors en el codi.

12.1. Tests sobre la infraestructura

Els punts principals que es vol comprovar mitjançant aquests tests és el funcionament dels serveis de Kinesis Data Stream i Kinesis Data Firehose per l'escriptura de dades i el servei d'Amazon Athena, conjuntament amb el servei d'AWS Glue, per la lectura de dades.

Pels tests d'escriptura es desenvolupa un script que permet l'enviament de dades al stream de Kinesis. D'aquesta manera podem comprovar a través de la consola d'AWS si el stream de Kinesis Data Stream va rebent dades. Això es pot observar en la figura 12.1. En aquesta figura podem observar la latència d'obtenció de les dades i també quina és la quantitat de dades que hi ha en el stream.

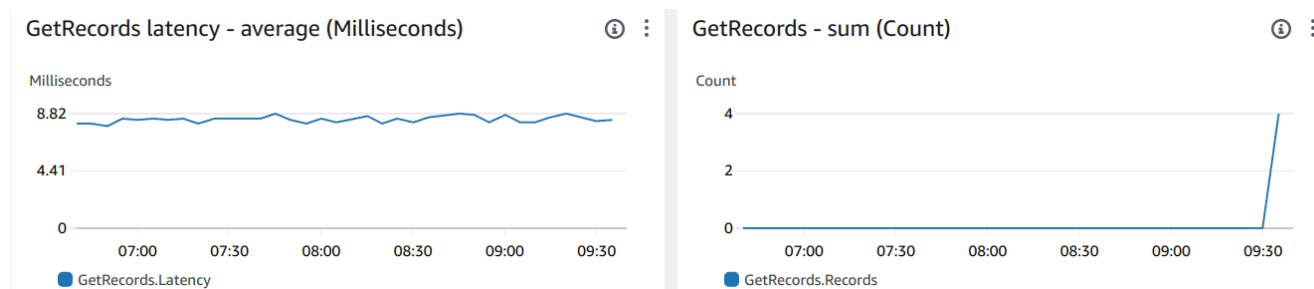


Figura 12.1. Dades de control de Kinesis Data Stream
Font: Consola d'AWS

Per acabar de comprovar l'escriptura, es comprova que Kinesis Data Firehose agafa les dades del stream i les emmagatzema en el bucket de S3 corresponent. Això es pot observar en la figura 12.2, on es pot veure que ha llegit totes les dades del stream. Després d'això comprovem que les dades es troben en S3.

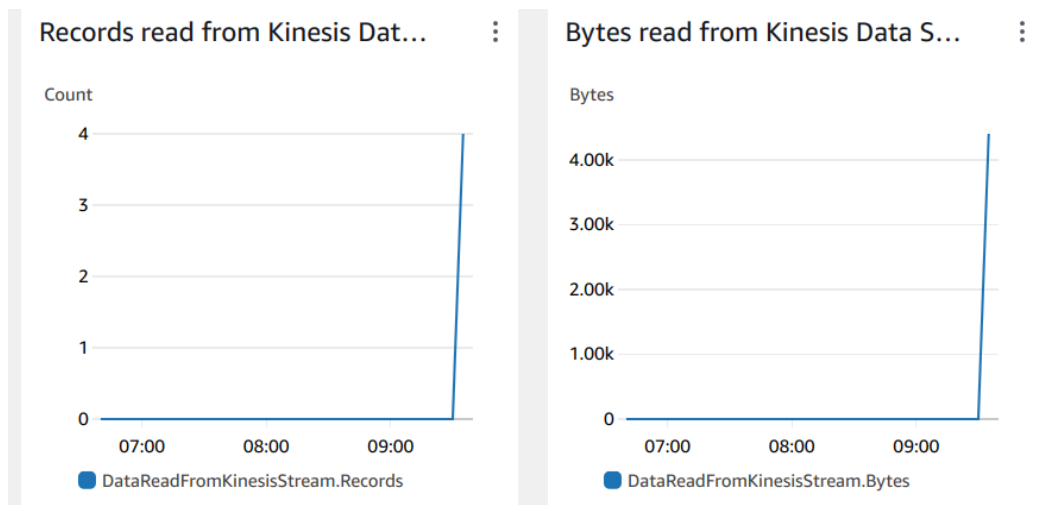


Figura 12.2. Dades de control de Kinesis Data Firehose
Font: Consola d'AWS

Pel que fa a lectura volem comprovar que les dades emmagatzemades en S3 poden ser llegides per Amazon Athena. Per poder dur a terme això fem ús de la consola d'Athena, que ens permet realitzar consultes SQL directament. En la figura 12.3 podem observar un exemple d'una consulta des de la consola. Si Athena està configurada de manera correcta ens retorna les dades de la mateixa manera que una base de dades.



Figura 11.3. Consulta d'Amazon Athena
Font: Consola d'AWS

12.2. Tests d'integració

Per poder assegurar el bon funcionament de les funcionalitats desenvolupades s'ha definit una llista de tests d'integració que s'han anat realitzant a mesura que es desenvolupava el projecte de manera manual.

Acció	Esriptura i obtenció de les dades d'audit trail
Descripció	Emmagatzemar una acció en la plataforma i visualitzar l'acció
Resultat	Èxit

Taula 12.1. Test d'integració 1
Font: Elaboració pròpia

Acció	Esriptura i obtenció de les connexions
Descripció	Emmagatzemar una connexió i visualitzar-la
Resultat	Èxit

Taula 12.2. Test d'integració 2
Font: Elaboració pròpia

Acció	Esriptura i obtenció dels playlogs
Descripció	Emmagatzemar playlogs d'un player i visualitzar-los
Resultat	Èxit

Taula 12.3. Test d'integració 3
Font: Elaboració pròpia